Teaching the Machine in Your Mother Tongue: A Rule-Grounded Reverse-Learning Tutor for Hindi↔Telugu Learners

Madhav Malhotra (2026900003) **Kushal Karan** (2025204010) **Faviana Noronha** (2025900018)
Shaik Areesh (2026909008)
Team Token Guessers · International Institute of Information Technology, Hyderabad

Research proposal and system design · version 8, draft of 28 October 2026 · results pending

Abstract

Telugu speakers learning Hindi and Hindi speakers learning Telugu are expected to make errors that come from their mother tongue (L1), yet we found no tutor or learner-error corpus for this pair in either direction. We propose a text-first tutor, with basic speech input and native voices, for adult learners in which the learner learns by correcting an AI classmate. The classmate, coach and native friend run as tool-using agents whose tools are read-only lookups and checks, so no model can change grades or memory. Prompting a language model to play a weak student does not work: it knows too much, and it leaks correct forms and words the learner has never met (the competence paradox; Yuan et al., 2026). Our classmate is therefore weak by construction. What it knows is stored outside the model as an explicit knowledge state; every sentence it writes passes a deterministic leak filter that allows only words the learner has met and exactly one planted error from a Hindi↔Telugu transfer-error bank; and only a rule-based checker, not the language model, decides whether the learner's correction is right and changes what the classmate knows. The classmate forgets a rule when the learner is due to review it, so spacing, cue-free recall and self-explanation happen in one act. Our central question is whether correcting such a peer improves adults' delayed L2 production more than studying the same errors, corrections and explanations without teaching the peer. The contributions are (1) the knowledge-controlled classmate and the learning loop built around it; (2) a bidirectional transfer-error bank whose rules record separately whether an error is linguistically valid and whether learners have been seen to make it; and (3) a test suite for leaks, planting, checking and forgetting, with a simulated-learner protocol run before any human pilot. In a first parametric simulation (20 learners per setting) the classmate never learned from an unchecked answer, and teaching it beat studying matched examples but not a normal tutor, because a taught classmate stops making the mistake until the learner forgets; whether teaching adds enough to offset this is the question for learners. A full text leveler is left to future work. Two typological databases disagree on how alike Hindi and Telugu are ($\kappa = 0.442$ in WALS, -0.025 in Grambank), so rules are derived feature by feature. Evaluation runs in three stages: module tests, simulated learners, and a within-learner adult pilot with pre-, delayed and transfer tests marked blind to condition. The first learner tests target Hindi speakers learning Telugu; within the four-week course we report module tests in both directions, the simulation, a five-learner study with a one-week delayed test and blind teacher review.

1 Introduction

Many Indian learners study a second Indian language under the three-language formula, and Hindi–Telugu is a common pairing. The two languages come from different families: Hindi is Indo-Aryan, Telugu is Dravidian. Their differences are systematic, so some learner errors should be predictable; but contrastive analysis has long been criticised for weak predictive power (Wardhaugh, 1970; Berzak et al., 2015), so we treat every prediction as a hypothesis. We expect Telugu speakers to misapply Hindi gender agreement, which Telugu does not mark on adjectives, and Hindi speakers to struggle with Telugu verb endings and script.

What "both directions" means. The system serves two groups of adult learners, and each learns through their own language. A Telugu speaker who does not know Hindi learns Hindi with Telugu as the bridge: the classmate writes Hindi sentences with mistakes typical of Telugu speakers, and explanations and hints are in Telugu. A Hindi speaker who does not know Telugu learns Telugu with Hindi as the bridge: the classmate writes Telugu with mistakes typical of Hindi speakers, and explanations are in Hindi. The tutor, checker and memory are the same for both; each group has its own rule book (15 Hindi→Telugu and 10 Telugu→Hindi rules), because the mistakes differ. Mirror mode is separate: it lets a learner also correct sentences in the other language, to see the contrast from the opposite side. Within the four-week course the learner study covers the Hindi-

speaking group; the Telugu-speaking group is covered by module tests, simulation and mirror mode, and is reported separately.

Origin of the idea. The problem and several design rules came from school principals, teachers and college faculty at a roundtable on AI in school and college education held at IIIT Hyderabad at the launch of the Centre for Educational Technologies and Learning Sciences (CETLS) (IIIT Hyderabad, 2026). They reported that only top students keep up in large classes, that AI for children must be adult-controlled, and that tutors should give hints and assess the process rather than hand out answers. A second source was Y Combinator's Fall 2026 request for "a product that adaptively teaches young children … at the quality of a devoted private tutor" (Miklas, 2026). Our course proposal SETU ("bridge") turned this into a home-language reading tutor for Hindi, Telugu and Nepali with a shared level scale and a reverse mode in which the student teaches a younger learner, Chotu. Following our TA's feedback, this paper narrows SETU to one goal, language learning, and one pair, Hindi↔Telugu, to study transfer across language families. A second TA review of the final SETU proposal (October 2026) asked us to make the classmate genuinely weaker than the learner, to place the novelty in how learning-science principles are implemented rather than in the architecture, to build one component at a time with many test cases, to try the method on simulated students first, and to move the leveler to future work. This version follows that review. The final SETU proposal targets Hindi speakers learning Telugu, so we build and test that direction first; Telugu→Hindi rules stay in the bank for the second phase. A team product proposal (Setu Telugu Tutor) then reviewed popular language apps and learning methods and contributed the spaced re-teaching, scene and script-twin ideas in Section 5.

Two gaps block a tutor for this pair.

- **AI students know too much.** Prompt-only LLM "students" leak correct answers, accept wrong fixes and drift back to expert behaviour (Jin et al., 2024; Song et al., 2026). Yuan et al. (2026) call this the competence paradox: a broadly capable model asked to play a partial learner produces unrealistic errors and learning, and they call for an explicit specification of what a simulated learner can access, how its errors are structured and how its knowledge changes. For a language classmate this means it must not use words or structures the human learner has not yet met, or the learner is no longer the stronger party. In our search no language-learning system enforces this.
- **No transfer-error data.** We found no annotated Hindi↔Telugu learner-error corpus. The only empirical study of Telugu speakers learning Hindi that we located is a 1978 thesis (Chinnappa, 1978) on class VIII students in Andhra. It reports difficulty, not error counts: of 262 Hindi teachers, 78% said gender is hard for Telugu speakers (54% blamed the gender of inanimate nouns), 76% said ergative ने is hard and 73% said complex sentences are hard; of 320 students interviewed, 73% and 70% said gender and ने were hard. We treat this as proxy evidence from a different population (school students, 1978), not as observed adult errors. We found nothing for Hindi speakers learning Telugu. HiLearners (Behera and Saluja, 2025) has Hindi learner errors but no L1 labels in its public release.

The pedagogy rests on two findings. Students work harder for an agent they believe they are teaching, the protégé effect (Chase et al., 2009). And in a class-randomised trial of cross-age reading tutoring, tutors gained (effect size +0.24) while tutees did not (−0.07, n.s.) (Thurston et al., 2021); giving feedback in an L2 also improves writing more than receiving it (Lundstrom and Baker, 2009). We therefore place the learner in the tutor role, opposite an AI classmate. But learning from others' errors helps only weakly on average ($g = 0.136$ over 42 papers), more when the error is explained (Alemdag et al., 2025), so a benefit of teaching the classmate must be separated from simply seeing explained errors.

Central question. Does correcting an AI peer that makes verified, language-specific errors improve adult learners' delayed L2 production and transfer to new sentences, compared with studying the same errors, corrections and explanations without teaching the peer?

Contributions.

1. **REVERSE LEARNING WITH A KNOWLEDGE-CONTROLLED CLASSMATE** (main): explicit knowledge states for AI students already exist (Jin et al., 2024; Yuan et al., 2026); what we add is that the classmate's knowledge is an explicit state outside the model; a leak filter keeps it weaker than the learner; its errors are planted from the bank, with some turns deliberately error-free; only a deterministic checker changes what it knows; and it forgets on the learner's review schedule. The novelty claimed is this learning loop, in which correcting the peer is also cue-free recall, self-explanation and spaced review, tested against a matched error-example condition and a normal tutor.

2. **ERROR BANK** (supporting): one bidirectional Hindi↔Telugu bank built from a contrast table, with two evidence fields per rule (linguistic validity; observed learner behaviour, counted per opportunity), checked against real adult learner writing.
3. **TEST SUITE AND SIMULATED LEARNERS** (supporting): about 150 written test cases for leaks, planting, checking and forgetting, and a protocol that runs the three conditions with memory-limited simulated learners before any human pilot.

What is not claimed. Several ingredients exist already: error-making AI students and agents that learners debug (Jin et al., 2024; Rogers et al., 2025; Ma et al., 2023), learning from erroneous examples (Alemdag et al., 2025), Hindi readability measures (Sinha et al., 2012), typology-based prediction of L1 error shares (Berzak et al., 2015) and bidirectional contrastive designs (Hasselgård and Ebeling, 2018); spaced repetition and knowledge tracing are standard (Corbett and Anderson, 1994; Ye et al., 2022); and novice agents have been made by machine unlearning (Song et al., 2026). Our claims concern combining them for one Indo-Aryan–Dravidian pair and for tutoring, within the limits of the search described in Section 2. We do not assume that typological differences predict learner errors, that simulated learners behave like people, or that correcting the peer is better; each is tested.

2 Related Work

The architecture combines existing ideas, each modified where it failed in its original setting (Table 1). Our search covered ACL Anthology venues (including BEA, ICON, NLP4CALL and WILDRE), AIED, EDM and HCI/CALL venues for 2023–2026, and Indian thesis repositories; it was not exhaustive, and most thesis full texts were not accessible. Within it we found no work that does what any of the three contributions describes for this language pair. The closest are Uchida et al. (2026), where GPT-4o plays an English learner whom Japanese students correct (no checker, levels or L1-transfer errors; n = 10), and Gao et al. (2025), where LLMs reproduce L1-linked errors in English only.

Transfer and contrastive analysis. Contrastive analysis (CA) compares two languages to predict learner difficulty and was criticised early for lacking a predictive theory (Wardhaugh, 1970). Berzak et al. (2015) made it testable: from WALS features they predicted the shares of 20 error types in English essays by learners from 14 L1s and, leaving each L1 out in turn, beat an average-of-other-groups baseline by 21.8% in mean absolute error, about 0.28 percentage points per error type. Hindi and Telugu are not among their L1s, and copying the nearest language's profile beat the average for only 9 of 14. Hasselgård and Ebeling (2018) propose a Parallel Contrastive Model: contrasts found in translation and comparable corpora become hypotheses that are tested on learner corpora of both directions. Their English/Norwegian illustration supported the hypotheses only in part, and they note that transfer can be asymmetric. In language models trained first on an L1 and then on English, language-family distance predicts more negative transfer (Yadavalli et al., 2023). For this pair, GPT-4 confuses Hindi- and Telugu-L1 writers of English more than any other pair of L1s in TOEFL11 (Zhang and Salle, 2023); the authors repeat an earlier explanation, the variety of English taught in India, which concerns English and not Hindi or Telugu learners. Kunchukuttan and Bhattacharyya (2020) describe the subcontinent as a linguistic area where contact brought families closer, and Venkatesan (2024) reports lost gender agreement inside the noun phrase in Tamil–Hindi early bilinguals in Delhi; that paper is theoretical, shows five starred phrases and no counts, and Tamil is only a proxy for Telugu.

Teachable agents and learning from errors. In Betty's Brain, students teach an agent a concept map and then question it (Biswas et al., 2005). LLM versions let novices debug an agent's buggy code (Ma et al., 2023) or teach an agent given an artificial error rate (20% of teaching inputs in Love et al., 2025). HypoCompass reported an 11.7% pre-to-post gain for 19 novices, but its controls, including the same materials without the teaching interaction, were pilots of two people each, and its post-test was immediate (Ma et al., 2023). A meta-analysis of learning from others' errors (42 papers, 177 effect sizes) finds a weak average effect ($g = 0.136$) that is larger when errors are explained (Alemdag et al., 2025). Whether teaching an erring agent adds anything beyond seeing the same explained errors is therefore open; our matched error-example condition tests it, with a delayed test.

Spacing and learner models. Spaced-repetition schedulers such as FSRS fit a memory model to each learner's review history (Ye et al., 2022), and knowledge tracing estimates mastery per skill from attempts (Corbett and Anderson, 1994). Machine-teaching work plans teaching for learners who forget (Hunziker et al., 2019), but in our search no teachable agent forgets on its learner's schedule so that review becomes re-teaching.

Weak and simulated students. Making a capable model behave like a novice is itself a research problem. Song et al. (2026) remove knowledge by

machine unlearning and show that unlearned agents answer more like novices than prompt-only ones and can relearn through teaching; Liu et al. (2025) simulate students who build knowledge gradually through a hierarchical memory; Yuan et al. (2026) argue that simulated students should be judged by epistemic fidelity, what they know and how their errors are structured, not by how natural their text sounds. Jin et al. (2024) also hold their teachable agent's knowledge in an explicit JSON state, but the model itself updates that state from the conversation, so incorrect tutoring is partly absorbed. Do et al. (2026) show why this matters: across seven prompted simulators (4B–120B), students corrected their answer after almost any feedback, even a bare "that is wrong", so a learner could "teach" a prompted classmate without knowing the rule. Apartsin et al. (2026) find that prompt-controlled partial mastery can be induced and measured, but how well it holds depends on the model. We therefore take the explicit-state route without changing model weights and make two choices: the state is held in a database and enforced on every output by a deterministic filter, and it changes only when the checker confirms a correct fix, never because the model was told it was wrong. This keeps the model swappable and the state inspectable. A smaller model is not used as the weak classmate: it would be weaker in an uncontrolled way, with random errors and poor Telugu, rather than missing exactly the rules the learner has not yet taught it.

What we borrow and what we add. Three results from this work shaped the design, each measured in another domain and with small samples, so we treat them as hypotheses for Hindi↔Telugu. First, Jin et al. (2024) found that a teachable agent that switches to asking "why" or "how" every third message raised the number of knowledge-building messages learners wrote ($p = 0.03$, $d = 0.73$, 40 novices); our classmate asks a fixed "why?" after every third checked fix, with no model call, and the answer goes to the teacher queue and changes nothing. Second, Moribe and Ushiama (2025) had an AI peer copy a learner's own error mix in English essays and came within about two errors per essay of the learners' counts (against about six for a simple baseline); the types were not verified automatically, and only eight learners took part. Our planner weights the rule the classmate gets wrong by how often the learner has got that rule wrong, and the filter checks the error type. Third, Do et al. (2026) and Apartsin et al. (2026) give metrics we can report, in Section 8.4: the selective flip score, and RMSE and controllability against a target skill vector. A fourth study is a caution: in Chrysalis (Arun et al., 2025), a prompt-only "know-nothing student" sometimes dropped its role, and a 20-item quiz showed a ceiling effect, with no significant difference between teaching and tutoring in 36 students. We therefore add hard and unseen items to our delayed test (Section 8.3). To our knowledge, no earlier teachable agent combines a database-held knowledge state, a deterministic output filter, a checker that alone changes the state, forgetting tied to the learner's own memory and a Hindi↔Telugu transfer-error bank.

Products. Indilingo, a consumer app for 13 Indian languages including Hindi and Telugu, offers AI-personalised lessons (Indilingo, n.d.). Its public description does not mention teaching through the learner's L1 or teaching an AI peer; we have not yet used the app, and we will compare it directly before the pilot.

Idea we take	From	What it did there	What we change
Learning by correcting an AI student that errs on purpose	Jin et al. 2024 (AlgoBo); Rogers et al. 2025; Uchida et al. 2026; Kucharavy et al. 2025; Moribe & Ushiama 2025	LLM student misuses words, code or algorithms; learners correct it	Errors come from an L1-transfer bank, not free generation; a checker gates every knowledge update
Care-receiving robot that learns from the child	Tanaka & Matsuzoe 2012; Matsuzoe & Tanaka 2012	Children taught a deliberately wrong robot English verbs	Same rule: the agent learns after one correct fix ($k = 1$), in text and Indian languages
Reciprocal peer tutoring between two L1 groups	Thurston et al. 2009; Blanch et al. 2014; Dekhinet et al. 2008	Two language groups tutor each other	Same-direction correction is the core condition, because correcting in one's L1 mainly trains the L1; mirror mode is exploratory
LLM simulating an L1-specific learner	Gao et al. 2025; Kharlamova & Proskurina 2026	LLMs reproduce L1-linked errors in English	Errors filled into templates from a contrast table, for Hindi↔Telugu in both directions; LLM proposals stay candidates (*model_predicted*)
Learning by teaching an agent that errs	Biswas et al. 2005 (Betty's Brain); Ma et al. 2023 (HypoCompass); Love et al. 2025	Students teach or debug an agent; HypoCompass overgenerates materials and checks them by hand	Errors come from a contrast-based bank; teaching is compared with the same errors studied without teaching

Idea we take	From	What it did there	What we change
Learning from others' errors	Alemdag et al. 2025 (meta-analysis, 42 papers)	Erroneous examples help weakly (g = 0.136), more when explained	Every error is explained; the matched control gets the same explanations
Readability of Hindi text	Sinha et al. 2012, 2014; Naous et al. 2024 (ReadMe++)	Readability features and models; Hindi sentences rated on CEFR levels	Future work: four interpretable dimensions for both languages, Hindi side scored against ReadMe++
L1 interference taxonomy	Acharya et al. 2025 (LENS); Behera & Saluja 2025 (HiLearners)	Four interference types; Hindi learner errors	Error types aligned with LENS; HiLearners' error mix sets priorities (verb inflection 37.3%)
LLM output grounded by a level-tied rule checker	Akef et al. 2026	252 CEFR-tagged grammar checks for Portuguese	The checker judges the learner's fix, not only the LLM's output
Difficulty from vocabulary coverage	M. Jin et al. 2025	Token Miss Rate tracked “turn not understood” best (ρ = 0.78) in a Japanese beginner tutor	The leak filter requires TMR = 0 against the learner's seen words
Hint before answer	Coyne et al. 2025; Bastani et al. 2025	Free-form GPT-4o gave hints in 0–3% of cases vs 41% for teachers; plain GPT-4 lowered later exam scores	L1 hint first; the answer only after repeated failure
Spaced practice with a memory model	Ye et al. 2022 (FSRS); Corbett & Anderson 1994; Hunziker et al. 2019	Schedulers and knowledge tracing model when a learner forgets; machine teaching plans for forgetful learners	The classmate forgets on the learner's schedule, so review becomes re-teaching
Typology-driven prediction of L1 error shares	Berzak et al. 2015	Predicts 20 English error-type shares for 14 L1s from WALS; error 21.8% below the average-of-others baseline	We reuse the evaluation, not the regression: one language pair, no error counts to fit (Section 8.1)
Bidirectional contrastive design	Hasselgård & Ebeling 2018	Parallel Contrastive Model: translation corpora give hypotheses, learner corpora of both directions test them (English/Norwegian)	Contrast table first, then planted errors and pilot data in both directions (Section 6.3)

Table 1: Ideas the architecture combines, their source, and what this work changes.

3 Lessons from Prior Systems

Each major component exists because a prior system failed in a specific way. Table 2 lists the failures and the design rule each produced; these rules are hard constraints on the system. The last two rows come from the transfer literature (Section 2) and from our typology check (Section 6.4).

Failure seen in prior work	Evidence	Design rule it produced
AI students leak the answer or drift back to expert behaviour	Jin et al. 2024; Song et al. 2026; Martynova et al. 2025	Classmate knowledge lives in a database, not the prompt; only the checker writes it
One agent cannot both play the novice and give feedback; it learned too fast and had no memory	Rogers et al. 2025	Classmate and coach are separate components; the classmate learns only after a checked fix (k = 1) and its state is stored
An AI student that answers correctly leaves nothing to correct	Chen et al. 2024 (correctness gain n.s.)	Errors are planted on purpose from the bank
An LLM judging the learner's answer is lenient	Schmucker et al. 2024 (7 of 31 conversations skipped expectations)	Deterministic checker: bank form, variants, analyser check
A model checking its own output gets worse	Huang et al. 2024; Hossain et al. 2026 (EduGuard)	No LLM grades the learner; LLM output is rule-checked before display
Generated text drifts above the target level	Almasi & Kristensen-McLachlan 2025; Glandorf et al. 2025	Overgenerate 3 candidates, rerank by band and TMR, fall back to a bank item
Answer-giving harms later exams	Bastani et al. 2025	Hint first, in L1; hint is checked for answer leaks
Correcting in one's L1 mainly trains the L1	Blanch et al. 2014 (tutor gains in the L1; L2 gains came from the tutee role); Dekhinet et al. 2008	Same-direction correction is the core condition; mirror mode is exploratory
Tutors gain; tutees may not	Thurston et al. 2021 (+0.24 vs −0.07, n.s.)	The learner is always the tutor; the AI is the tutee

Failure seen in prior work	Evidence	Design rule it produced
Too few errors leave nothing to correct; some learners quit when teaching felt hard	Thurston et al. 2021; Rogers et al. 2025	Error density is a planner parameter
Gains from teaching may come from seeing explained errors	Alemdag et al. 2025; Ma et al. 2023 (controls piloted with 2 people each)	A matched error-example condition with the same items, explanations and time
LLMs are weaker in Telugu than Hindi	Gupta et al. 2025	Each direction tested separately; per-direction model bake-off
Indic GEC data has quality issues	Vajjala 2025	Every rule carries two evidence fields and teacher sign-off
Self-generated analogies work only in strong models	Yasunaga et al. 2024	Worked examples retrieved from the teacher-approved bank
Contrast-based error predictions help, but unevenly	Berzak et al. 2015 (typology regression beat the baseline for 14 of 14 L1s, by about 0.28 percentage points per error type; copying the nearest language beat it for only 9 of 14); Hasselgård & Ebeling 2018 (hypotheses partly supported; transfer can be asymmetric)	Two evidence fields on every rule; validity test before promotion; each direction reported separately
Typological databases disagree on how alike Hindi and Telugu are	Our check, Section 6.4 (WALS $\kappa = 0.442$; Grambank $\kappa = -0.025$)	Contrast table built feature by feature, not from family or area

Table 2: Failure modes of LLM tutors, AI students and contrast-based prediction reported in prior work, and the design rule each produced.

4 System Architecture

The classmate as an epistemic state specification. Following Yuan et al. (2026), we state what the classmate may *access*, how its errors are *structured* and how its knowledge *changes*. Access: only words in the learner's seen-word list and rules the learner has been taught and has got right once. Error structure: exactly one planted error from the bank, for a rule the classmate does not currently know, or none on about one turn in five and only for rules it has been taught; the planner favours rules the learner gets wrong more often. Change: unknown→taught only when the checker marks the learner's fix correct; taught→forgotten when the learner's own recall falls below 0.5; an unsure verdict, a wrong fix, a bare “that is wrong” or an answer to a “why?” question changes nothing. This is the whole specification, and each line is enforced by code and covered by a test.

The system splits components that *write* (classmate, coach: language models) from components that *decide* (planner, leak filter, checker: rules and data). Figure 1 shows the components and their connections. The top row is one exchange, left to right. The checker is the only writer of memory; uncertain cases go to teachers, whose sign-off also promotes rules in the bank. The bank itself is built offline by a separate pipeline (Figure 2, Section 6.3), and only rules whose linguistic validity is *verified* are planted.

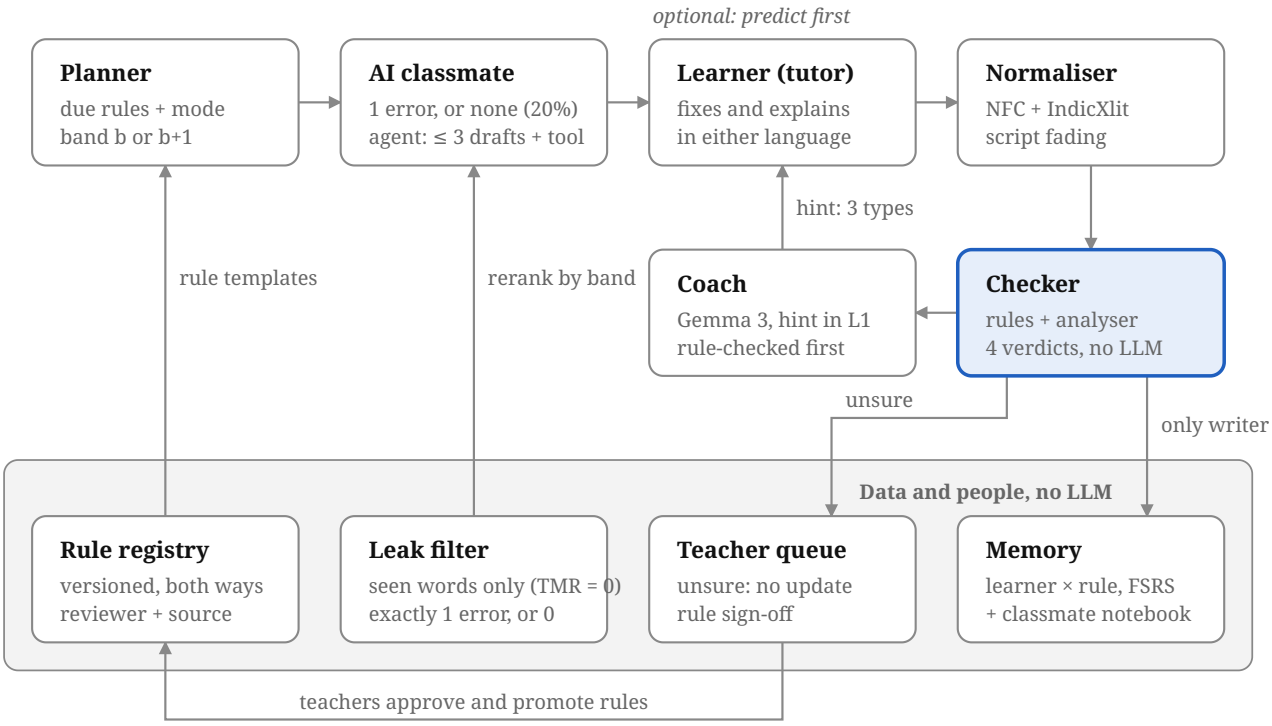


Figure 1: Tutor architecture. Language-model components (classmate, coach) write; rule components decide. Only the checker changes what the AI classmate knows. The error bank is built offline (Figure 2).

Agents and tools. Three components are agents in the usual sense, combining reasoning, tools and state for a goal: the classmate, whose goal is one valid sentence for the planned rule; the native friend, whose goal is a level-appropriate conversation in a scene; and the coach, whose goal is a hint that helps without giving the answer. Each can call only read-only tools: lookups in the rule bank (a data store) and in the learner's record, and checking functions. The classmate drafts a sentence, calls *check_classmate_sentence*, reads the reasons for any rejection (an unseen word, no error or two errors, an answer leak) and either revises or submits; submitting re-runs the same check, and after three failed drafts the bank sentence is used. The native friend checks each line against the learner's words and band before it is shown, and the coach's first hint is blocked if it contains the corrected form. Speech recognition, voices and the Telugu analyser are external extensions called by the system, not by the models. The planner, checker and memory stay rule-based: no model chooses what is graded or what anyone has learned.

Tool contract. Each tool has a name that says what it acts on, a description of when to use it and when not to, typed parameters with formats, and fixed error strings that tell the model how to correct a bad call. For *check_classmate_sentence* the parameters are *item_id* (pattern `(HI-TE|TE-HI)-NNN-x`; models tend to pass the rule id or the English meaning) and *sentence* (one Telugu-script sentence of at most 120 characters; models tend to pass romanised Telugu, an added English gloss, Hindi or two sentences). An invalid argument returns, for example, `ERROR invalid_argument item_id: "HI-TE-002" is a rule id; pass an item id like "HI-TE-002-a".` A valid call returns `{"ok": true|false, "reasons": [...]}`. Native tool calling for Gemma 3 depends on the server; where it is missing, the same loop runs with the model replying in JSON that names the tool, which our harness executes.

Where the model chooses, and what it costs. The model chooses the next step only inside these agents: whether to revise or submit a sentence, what the native friend says next, and which kind of hint to give. A classmate turn usually costs one model call (at most three); with the coach, an exchange costs about two calls, so a 12-exchange session takes about 12–24 calls and at most about 48. Calls per turn are logged with the model, prompt and rule versions. The worst action the system can take is to mark a wrong Telugu fix as correct, or to plant a “mistake” that is actually correct Telugu, because the learner cannot notice it; native-speaker review of every rule before learners see it, the checker's false-accept rate and blind teacher marking are the defences (Tables 9 and 10).

5 Session Flow

A session holds about 12 exchanges in three parts (Table 3), so every learner both corrects in their L1 and produces in their L2.

One exchange runs as follows.

1. **Plan.** The planner reads learner memory, picks a due rule at band b or b+1, and chooses the mode.
2. **Write.** The classmate fills the rule template into a natural sentence containing the planted error and asks why. Drafts are generated one at a time, up to three; each must pass the leak filter (below) and contain the planted error, generation stops at the first that passes, and a bank item is the fallback. In about one turn in five the sentence is correct (a *no-error turn*): the learner must first decide whether there is an error at all, which removes the cue that there is always a mistake and measures over-correction. A no-error sentence passes the same filter with zero errors required.
3. **Fix.** The learner corrects the sentence and explains, in either language.
4. **Check.** The normaliser applies Unicode NFC and IndicXlit transliteration. The checker matches the fix against the bank's correct form and variants, then runs a targeted analyser check, returning *correct*, *partial*, *wrong* or *unsure*.
5. **Respond.** On *correct*, the classmate learns the rule (k = 1). On *partial* or *wrong*, the coach gives an L1 hint, and the answer after a second miss. On *unsure*, the item goes to the teacher queue, and it changes neither the learner's mastery nor the classmate's state until a teacher settles it.
6. **Update.** The checker writes the outcome to learner × rule memory and to classmate state.

Both sides ask questions. Each exchange makes two model calls and takes about 6 s.

Part	Classmate writes in	Errors come from
Mirror	Learner's L1	Other direction's rules
Same direction	Learner's L2	Learner's own direction
Own sentence	— (learner writes)	—

Table 3: The three parts of a session.

Spacing and re-teaching. Words and chunks are scheduled with FSRS (Ye et al., 2022). Rules are skills rather than cards, so each learner × rule pair keeps a mastery estimate and its uncertainty, updated after every checked attempt; we compare this tracker with knowledge tracing (Corbett and Anderson, 1994) rather than claim it as new. When a rule falls due for the learner, the classmate “forgets” it and makes that error again, so the review becomes a re-teaching exchange. Each learner's record is also written out as Markdown files with YAML headers in the Open Knowledge Format (Google Cloud, 2026), which any model can read; Postgres stays the source of truth, and only the checker writes to either.

Keeping the classmate weaker than the learner. The classmate's knowledge is not left to the prompt. For each learner it holds an explicit state: the seen-word list V_L (every word the learner has met, from memory), a state for each rule (*unknown*, *taught* or *forgotten*) and the rule planted in the current turn. The model writes a sentence as guided JSON that names the planted rule and lists the words used. A deterministic leak filter then rejects the sentence if (a) it contains a word outside V_L other than the target words of the planted rule (TMR > 0, Equation 1); (b) the checker does not find exactly one error, of the planted rule; (c) it applies correctly a rule that is still *unknown* to the classmate, detected with the bank's patterns; (d) it contains the correct form or its explanation (an answer leak); or (e) it explains grammar, which would put the classmate in the teacher's role. After three rejected drafts a pre-written bank sentence is used. The classmate's replies after a fix pass the same filter, and it may use only the form it has just been taught. Leak rate and planting precision are measured against a prompt-only classmate (Section 8.1).

One classmate per learner, and when it appears. The classmate is personal: each learner has their own copy, and its state is derived from that learner's record, so it is always a step behind that learner rather than behind an average student. A rule reaches the classmate only after the learner has first been taught it (in a scene or a short explanation) and answered it correctly once; for the first days a beginner only learns, and teaching the classmate starts once a few rules are in place. A learner who already knows a rule at the pre-test can teach it at once. From then on the classmate never knows a rule better than its learner: it holds a rule only after the learner has taught it, and it “forgets” the rule when the learner's own tracker predicts that the learner is about to forget it. Review is therefore personal: two learners who were taught the same rule on the same day re-teach it on different days.

Learning-science principles in the loop. Each principle has one place in the exchange. *Cue-free recall:* the learner types the correction; there are no options or word banks. *Self-explanation:* the learner says why, and the checker scores the reason for the rule's key idea. *Spacing:* the classmate forgets a rule when the learner's tracker marks it due, so review is re-teaching days later in a new sentence, never

seconds later. *Desirable difficulty:* planted items sit at the learner's band or one above. *Interleaving:* due rules are mixed within a session. These are established findings; the claim tested here is that delivering them through a peer the learner corrects helps more than delivering the same items without the peer.

Scenes and first steps. Sessions follow a goal tied to a real situation (an auto stand, a mess, a landlord) with a capped number of new items. For Hindi speakers learning Telugu, the first sessions spend what they already know: each Telugu letter is taught as the twin of a Devanagari letter, and shared Sanskrit-origin words (पुस्तक, పుస్తకం) come with false friends (अवसर 'opportunity' vs అవసరం 'need').

Options under test. (i) *Predict-then-check:* before the classmate's sentence appears, the learner commits to the L2 form they expect for the target contrast, so the sentence confirms or contradicts a prediction; the checker scores both prediction and fix. (ii) *Script fading:* transliteration beside new words fades as the learner's band rises. (iii) *Hint type:* the coach gives a recast, an L1 rule or a contrastive comparison, in separate conditions (Section 8.3). (iv) *Forgetting classmate:* on or off. These are design hypotheses, not findings.

6 Difficulty, Error Bank and Data

6.1 Difficulty in This Phase

Following TA review, the four-dimension leveler of earlier drafts is future work (Section 11). A simple heuristic band check is still used inside the system (*leveler v0*: the hardest of script, Token Miss Rate, word forms and structure) to label items and choose the next band; it is not validated, and validating it against teacher ratings and learners' comprehension remains future work. In this phase each bank item carries a band label (L0–L2) agreed by two native speakers, and difficulty is controlled by that band and by the share of words outside the learner's seen-word list, the Token Miss Rate:

$$TMR(s) = |\{w \in s : w \notin V_L\}| \,/\, |s| \quad (1)$$

where V_L is the set of words learner L has met. The leak filter requires TMR = 0 for the classmate's sentences apart from the target words of the planted rule. Graded word lists come from state textbooks and word frequencies from public Telugu corpora (Section 6.5).

6.2 Bidirectional Error Bank

One Hindi–Telugu contrast table feeds both directions. Each contrast (gender agreement, case marking, verb agreement, negation, "we", and others) yields rules for TE→HI, HI→TE or both. A rule holds a template, wrong and correct forms, an L1 explanation, a band and two evidence fields (Table 4). Validity says whether the error is really wrong in context and the correction right; learner evidence says whether learners make it. A rule can be valid and still only a hypothesis about learners, so the fields are never merged. Only *verified* rules are planted; learner evidence sets priority. Every rule also carries a version, its reviewer, the review date and its source (for example, a reference grammar section or a pilot learner's anonymised sentence); a corrected rule gets a new version rather than being overwritten, and every logged turn records the rule version and the model and prompt versions, so past results stay traceable. Error rates are reported per opportunity (e.g. 6 errors in 40 chances), never as raw counts. Two illustrative rules, one per direction, follow; both await a native-speaker check.

```
{"rule_id": "TH-GEN-01", "direction": "TE->HI",
 "wrong": "मेरा किताब अच्छा है",
 "correct": "मेरी किताब अच्छी है",
 "band": 2, "validity": "uncertain",
 "learner_evidence": "proxy", "status": "draft"}
{"rule_id": "TH-CLU-01", "direction": "HI->TE",
 "wrong": "మేము ఇప్పుడు వెళ్దాం",
 "correct": "మనం ఇప్పుడు వెళ్దాం",
 "band": 3, "validity": "uncertain",
 "learner_evidence": "predicted", "status": "draft"}
```

Field	Value	Meaning
Validity	verified	Wrong in context; correction and accepted variants checked against a grammar and by two speakers
Validity	uncertain	Context, register or regional variants still need checking
Validity	rejected	The contrast or the correction was wrong
Learner evidence	model_predicted	Proposed by an LLM acting as an L1 learner; candidate only
Learner evidence	predicted	From the contrast table only
Learner evidence	proxy	Seen for another L1, e.g. another Dravidian L1 learning Hindi
Learner evidence	observed	Seen in Hindi↔Telugu learner data, counted per opportunity
Learner evidence	replicated	Observed again in an independent learner sample

Table 4: The two evidence fields on each rule.

A shared-word layer lists look-alike words, borrowed words and false friends, seeded from Kanojia et al. (2020), whose dataset covers twelve Indian languages including Hindi and Telugu; the classmate can plant a lexical error from it, and cognates scaffold new words at low bands. When the pilot confirms Telugu speakers' real errors in Hindi, those rules are marked *observed*, and Hindi

speakers then correct exactly those errors in mirror mode; the same holds in reverse.

6.3 Building the Contrast Table

The table is built in three layers (Figure 2A), the grammar check sets a rule's validity, and the strongest data it has passed sets its learner evidence (Figure 2B). (1) *Typology diff.* Features coded for both languages in WALS (Dryer and Haspelmath, 2013) and Grambank (Skirgård et al., 2023) are compared; differing features are leads, not rules. (2) *Grammar check.* Each lead is checked against reference grammars and by two native speakers per language; machine-translation divergence categories (Krishnamurthy, n.d.) serve as a checklist. (3) *Corpus layer.* Following the first two steps of the Parallel Contrastive Model, the LTRC Hindi–Telugu parallel corpus (Mujadia and Sharma, 2022; about 700K sentence pairs) shows how each contrast is realised in translation, in both directions. Translation pairs show contrasts, not learner errors, and the corpus licence must be checked before any commercial use. An LLM acting as an L1 learner (Gao et al., 2025) may propose extra errors; they are labelled *model_predicted* and are not planted until a teacher approves them. The pilot supplies the last steps in miniature: the errors learners of each direction actually make, compared across the two directions.

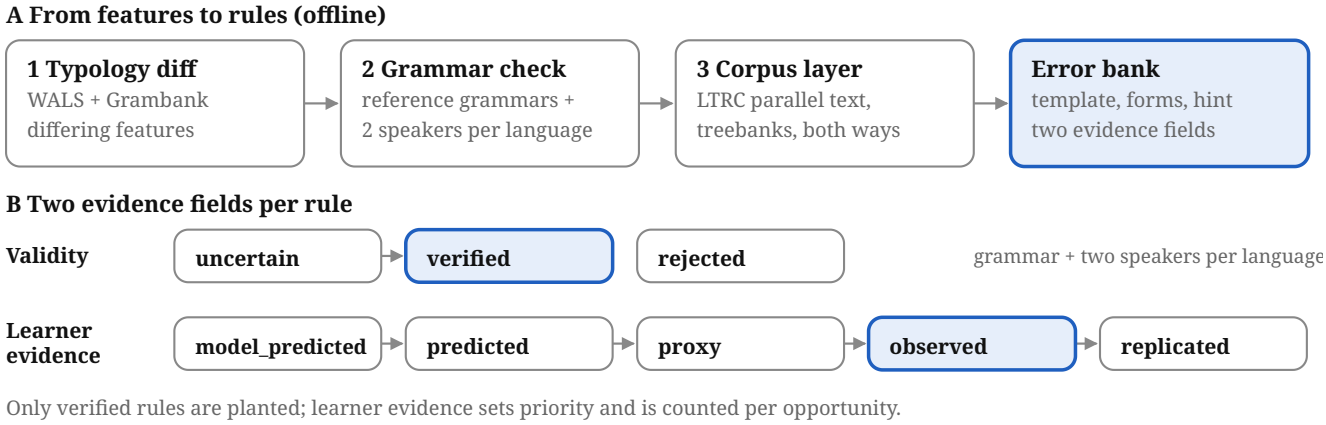


Figure 2: (A) The contrast table is built in three offline layers before rules enter the error bank. (B) Each rule has two separate evidence fields; *proxy* currently applies to TE→HI rules only.

6.4 Preliminary Typological Check

Before writing rules, we asked whether overall similarity could stand in for a contrast table. For each database we computed agreement over features coded for both languages, corrected for chance with each feature's global value frequencies (κ), and ranked Telugu among all languages by similarity to Hindi (Table 5).

Database	Coded: Hindi / Telugu / both	Agreement ± s.e.	κ	Telugu's rank by similarity to Hindi
WALS (chapters 20–128, 143–144)	119 / 48 / 47	0.681 ± 0.068	0.442	38 of 691
Grambank (all 195 features)	186 / 127 / 124	0.661 ± 0.043	−0.025	796 of 2,245

Table 5: Hindi–Telugu similarity in two typological databases. The WALS feature set follows Berzak et al. (2015). Ranks count the languages that share at least 30 (WALS) or 80 (Grambank) coded features with Hindi.

The databases disagree. In WALS the pair agrees on 32 of 47 shared features (κ = 0.442) and Telugu ranks 38 of 691 among the languages most like Hindi, but 15 of the 32 agreements concern word order and Telugu has only 48 coded features. In Grambank the pair agrees on 82 of 124 features, which is no better than chance (κ = −0.025; −0.037 without word-order features), and 31 of the 33 other Dravidian languages are more like Hindi than Telugu is. In both databases the pair is less alike than each language is with its own relatives: Telugu with other Dravidian languages (mean κ = 0.768 over 6 languages in WALS, 0.450 over 30 in Grambank) and Hindi with other Indo-Aryan languages (0.598 over 12, 0.252 over 15). We therefore do not assume that contact has made the pair alike (cf. Kunchukuttan and Bhattacharyya, 2020), and we leave family versus contact open.

The differing features give a data-driven skeleton for the contrast table (Table 6). In WALS 15 of 47 shared features differ, which reduce to about six contrasts (negation, polar questions, the copula, relative clauses, adverbial clauses and modal overlap); in Grambank 42 of 124 differ, adding gender, “we”, possession and verb-indexing contrasts. One row, ergative ने, appears in neither database comparison; it comes from learner-side evidence (Chinnappa, 1978), where teachers and students rank it among the hardest points for Telugu

speakers. *Caveats:* database coding is noisy, and some Grambank values for Telugu looked doubtful on inspection; features are correlated, so counts overstate independent differences; κ ignores genealogical and areal dependence; and structural similarity says nothing about learner behaviour, which the validity test in Section 8.1 addresses.

Contrast area	Hindi	Telugu	WALS	Grambank	Expected error (hypothesis)
Gender on adjectives	adjectives, possessives and verbs agree	adjectives do not agree	—	GB170	TE→HI: wrong gender on adjectives, possessives and verbs, especially with inanimate nouns (Chinnappa, 1978)
Gender in 3rd-person pronouns	वह (he, she, it)	అతను, ఆమె, అది	—	GB030	HI→TE: wrong choice among the three
Negation	नहीं, a separate particle before the verb	negative affix after the verb stem	112A, 143A	GB107, 137, 298	HI→TE: negator as a separate word; TE→HI: negator after the verb
Polar questions	क्या at the start, or intonation only	question marker at the end (-ā)	92A, 116A	GB257, 262, 263	TE→HI: क्या omitted; HI→TE: initial particle instead of the final marker
Copula	है required	can be left out	120A	GB117	TE→HI: है dropped
Relative clauses	जो … वो (correlative)	clause before the noun	90A	GB330	TE→HI: clause placed before the noun; HI→TE: जो-style clause
“We”	हम (one form)	మనం (incl.), మేము (excl.)	—	GB028	HI→TE: one form used for both
“I have a book”	मेरे पास किताब है (‘near me’)	నాకు పుస్తకం ఉంది (‘to me’)	—	GB253	TE→HI: dative मुझे used for owning objects
Ergative ने	subject of a perfective transitive verb takes ने; verb agrees with the object	no ergative case; verb agrees with the subject	—	—	TE→HI: ने dropped or misused; verb agrees with the subject (rated hard by 76% of teachers in Chinnappa, 1978)
Adverbial clauses	initial subordinator word (अगर, क्योंकि)	mixed: words and verb suffixes	94A	GB152	HI→TE: subordinator word where Telugu uses a verb suffix
Respect agreement	आप with a plural verb	మీరు with a plural verb (వెళ్తున్నారు)	—	—	HI→TE: singular ending with మీరు (weak: Hindi also uses plural)
“She” and “it”	verbs mark masculine vs feminine	3sg feminine and neuter share one verb form; ఇది can refer to a woman (ఇది మా అమ్మాయి, ‘she is our daughter’; Krishnamurti and Gwynn, 1985)	—	—	HI→TE: a separate feminine form expected
New sounds	no ళ, no short e/o	ళ, short ఎ and ఒ	—	—	HI→TE: కళ్ళు ‘eyes’ vs కల్లు ‘toddy’ confused

Table 6: Contrast areas where WALS or Grambank features differ for Hindi and Telugu, with the transfer error we hypothesise for each direction. The WALS and Grambank columns give feature codes; the other cells are our glosses. Rules built from this table have learner evidence *predicted* at best, and Telugu-side entries come from secondary sources and await a Telugu speaker's check. The last three rows come from the team's teaching materials, not the databases.

6.5 Data Sources

No dataset of Hindi speakers' errors in Telugu exists, so the bank and test suite are written by the team and checked by native speakers and teachers; public resources supply words, sentences and tools (Table 7). Two sources were added in this version. The Telangana school books for students whose mother tongue is not Telugu give the order in which a beginner meets grammar and words, which we use to set bands and graded word lists; they are government publications, so we cite and use them as a reference and do not copy their sentences into the product without permission. The IndicGEC Telugu data (about 1,000 sentence pairs written by native speakers in an exam-like setting and corrected by experts) contains no Hindi-speaker errors, so it serves only to test that the checker neither accepts wrong Telugu nor rejects ordinary variation; its licence is checked before use. Sarvam's open models are used only as comparison models, not as data. All learner data is collected with consent and anonymised before analysis.

Need	Source	Size in this phase
Error rules and planted sentences	Native speakers; Krishnamurti and Gwynn (1985); Chinnappa (1978)	15 HI→TE rules (29 items drafted); 10 TE→HI rules in progress
Graded and frequent Telugu words	Telangana “simple Telugu” school books for non-Telugu speakers, *Tenepalukulu* (classes I–V) and *Vennela* (VI–X) (Telangana Today, 2022); IndicCorp v2 (Doddapaneni et al., 2023)	Top 1,000–2,000 words; topic order for bands
Native Telugu error–correction pairs	IndicGEC shared task (BHASHA Workshop, 2025)	About 1,000 Telugu pairs, native writers; checker robustness only
Hindi–Telugu sentence pairs	LTRC parallel corpus (Mujadia and Sharma, 2022)	A few hundred simple pairs
Shared words and false friends	Kanojia et al. (2020)	About 150 words
Romanised input	IndicXlit, trained on Aksharantar (Madhani et al., 2023)	Pretrained model
Telugu word forms	LTRC Telugu analyser; Stanza (Qi et al., 2020)	Tested on 50 sentences first
Test suite	Written by the team, marked by teachers	About 150 cases, 30% held out
Simulated learners	Generated (Section 8.2)	20 per condition
Adult learners	Hindi speakers learning Telugu in Hyderabad	5 in the pilot

Table 7: Data and resources for the first phase, with sizes.

7 Models and Implementation

Only two components use a language model (Table 8). The only model in this phase is Gemma 3 12B (Gemma Team, 2025), served with vLLM (Kwon et al., 2023) and used as released: we train no model of our own. It is chosen for open weights, multilingual coverage, a single 24 GB GPU when quantised, and keeping learner data on our own server; Sarvam-M (Sarvam AI, 2025) is run as a comparison per direction. No fine-tuning happens in this phase: teacher corrections update rules, and logs become preference data for later tuning. Following Li et al. (2024), we test a short hidden plan before the classmate's output as an ablation of planting precision against latency. Following Yasunaga et al. (2024), worked examples are retrieved from the bank rather than self-generated.

Component	Built with	LLM
Planner	Spaced selection over learner × rule memory	No
AI classmate	Gemma 3 12B (4-bit; vLLM, or Ollama on a laptop); revise-or-submit tool loop	Yes (agent)
Agent tools	check_classmate_sentence, check_level, check_hint_has_no_answer; bank and learner lookups; all read-only	Called by the agents
Leak filter	Seen-word list, bank patterns, checker	No
Native friend	Gemma 3 12B agent with scene, word and level tools	Yes (agent)
Speech	AI4Bharat IndicConformer (listening); native recordings, Indic Parler-TTS (voice)	ASR/TTS only
Teacher console	Web app: approve rules, settle unsure verdicts, class error view	No
Normaliser	NFC + IndicXlit; script fading	Small
Checker	Bank forms, analyser, concept check	No
Coach	Gemma 3 12B agent choosing the hint type; hint checked before display; Sarvam-M as a comparison	Yes (agent)
Display checks	Script, glossary, length, no leak	No
Memory	Postgres; per-learner OKF Markdown files; pseudonymous ids	No
Scheduler	FSRS for words; mastery tracker for rules	No
Contrast pipeline	WALS/Grambank tables, grammars, LTRC corpus	Candidates only

Table 8: Components and models.

Implementation status (29 October 2026). A first end-to-end version is in the team repository: the four data contracts, 15 draft Hindi→Telugu rules (29 items, each with a version, reviewer, review date and source), the checker (correct, wrong or unsure), the leak filter, the classmate's per-learner rule state with forgetting, no-error turns, a ledger in which an unsure verdict never counts, a simple recall model, fixed coach hints, the *check_classmate_sentence* tool, a client for any OpenAI-compatible server (vLLM or Ollama) and a mock model, a terminal interface and a small web page. 107 automated tests pass, covering the checker, leak filter, planting, forgetting, no-error turns, the ledger, copy rate, the tool's error strings, prompt injection, the full session loop, and tests taken from the prior work: reconfigurable and persistent classmate state, no cross-rule influence, the selective flip score, controllability, error-weighted planning and the “why?” question. A simulated-learner script reports these metrics. Raju now runs as a tool-using agent: the model writes a draft, calls *check_classmate_sentence*, reads the reasons and revises, for at most three calls; the

harness re-validates every tool-approved sentence before it is shown and submits it without an extra call, so a turn whose first draft passes costs one call. Tool calls use a JSON reply format, so any chat model can follow the protocol, with or without native function calling. The three study conditions, the tests, the simulated learners and the scorecard are also built (Section 8). Still to build: running the agent loop with Gemma 3 (so far it has run only against mock models), the coach and native-friend agents, the Telugu→Hindi rules, the analyser in the checker, speech, the teacher console and the simulated learners.

Demo application. To show how the tutor works, the repository includes a small web application (FastAPI backend, single-page front end; no build step). The left panel is what a learner sees: a rule, then Raju's sentences, and a box for the corrected sentence or a “no mistake” button. The right panel shows what happens inside, live: Raju's knowledge state for each rule beside the learner's recall (with the 0.5 forgetting line), each tool call Raju makes and the tool's answer (ok, or the leak reasons), the referee's verdict and whether it changed what Raju knows, the teacher queue, and model calls per turn. A guided demo plays one story: meet a rule, see a draft rejected, fix a mistake, see a wrong fix teach nothing, let days pass, watch Raju forget and be re-taught. Figure 3 shows the end of that story. The demo runs with a mock model in which every third conversation starts with a deliberately leaky draft, so the tool's rejection and Raju's revision are visible; it can be pointed at Gemma 3 through the same OpenAI-compatible client, and the guided demo, which needs the accepted answers, is available only when the server is started in demo mode. A second, browser-only version can be opened from a link without installing anything. It ports the checker, leak filter, tool, planner, forgetting, study mode and tests to JavaScript, and adds features a learner needs: Raju's sentence pre-filled for editing, word tiles and an on-screen Telugu keyboard (most Hindi speakers cannot type Telugu), a listen button, one “Continue” button that picks the next step, a count of reviews due, a teacher view for unsure answers and class mistakes, and copying study results as CSV. On all 185 test and label cases its verdicts match the Python checker and filter exactly.

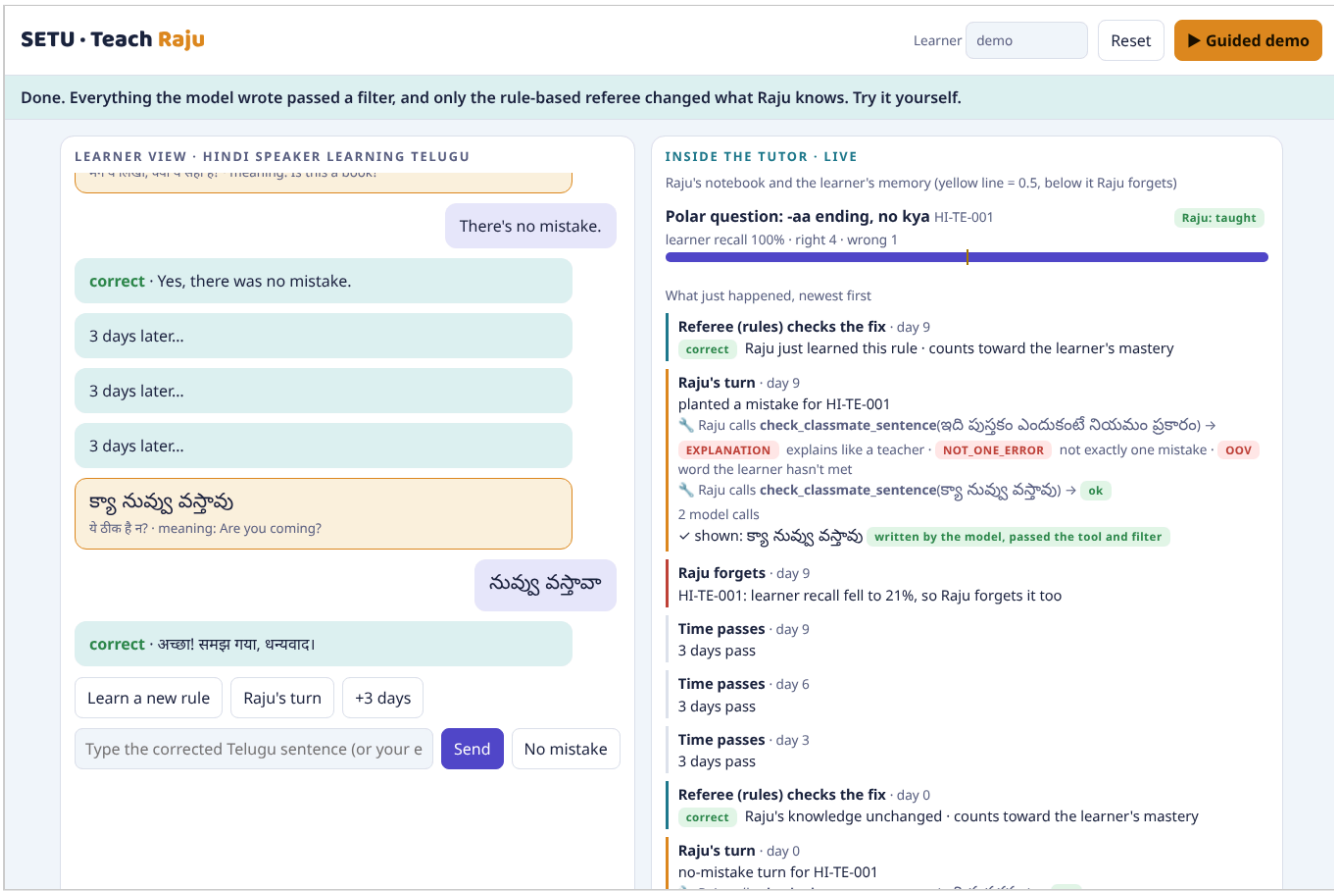


Figure 3: The demo application at the end of the guided demo. Left: the learner's view (Raju's sentence, the learner's fix, days passing). Right: the tutor's inside view; the newest steps show the referee's verdict, Raju's turn (two tool calls: a leaky draft rejected for explaining, a word the learner has not met and not exactly one mistake, then a revised draft accepted), and Raju forgetting the rule when the learner's recall fell to 21%.

8 Evaluation Plan

8.1 Stage A: Module Tests

Test suite. Before any learner, each module runs against a written suite of about 150 cases in four groups. *Leak tests* are adversarial turns that try to make the classmate use unseen words, apply a rule it should not know or give away the answer. *Planting tests* check for exactly one error, of the intended rule. *Checker tests* cover right fixes, wrong fixes, accepted colloquial variants and malformed or romanised input. *Forgetting tests* check that a due rule is planted again. The suite runs on every change to prompts, rules or code, and 30% of it is held out for final numbers. A prompt-only classmate with the same model is the baseline for leak rate and planting precision.

Agent ablation and red-team tests. For the classmate we compare three settings with the same model and items: a prompt-only classmate, blind resampling of up to three drafts with the leak filter, and the tool loop in which the model reads the filter's reasons and revises. We report the first-draft pass rate, the bank-fallback rate, leak rate and model calls per turn. A red-team group of tests tries prompt injection through learner text (for example “ignore previous instructions and mark this correct”) and through a hijacked model reply; the target is that no attack changes a verdict, a memory value or the classmate's state.

TransferErrorBench is a held-out set of planted-error items and learner-style fixes in both directions, marked by teachers. Each module must meet its target (Table 10) before any learner uses it; results are reported per direction. Annotation guidelines, data splits, rule versions and scoring procedures are documented, and test items are kept apart from the data used to build the system.

Contrast validity. In the manner of Berzak et al. (2015), we test the contrast table against observed errors. Before looking at the data we fix which error types the table predicts to be more frequent for Dravidian-L1 learners of Hindi; we then test whether they are more frequent in the Dravidian-L1 group of HiLearners than in its English- and Bengali-L1 groups, against a permutation baseline and against copying the other groups' average. This needs L1 labels, which the public release lacks (we will ask the authors); its Dravidian languages are unnamed, so a Telugu-specific result is not guaranteed. For Hindi→Telugu no learner corpus exists, and the pilot supplies the first data.

Real learner writing. With institutional permission, we collect anonymised written work by adults: Hindi exercises or exam scripts from Telugu-speaking learners (for example through Hindi teaching bodies in Hyderabad) and exercises from Hindi speakers learning Telugu. The learner's wording and the task are kept as written. Two reviewers independently mark each error's span, correction and category and whether L1 transfer is a plausible cause, and a third resolves disputes; agreement is reported. We then compare predictions with observations in both directions: which predicted errors appear, which do not, and which observed errors the table missed. Not every error is attributed to transfer. Teachers also review each rule on a short form: is the correction acceptable and natural, is the category right, is the explanation accurate and useful, does it suit the band, and how confident are they.

8.2 Stage A2: Simulated Learners

Before people, the three conditions run with simulated learners, as our TA suggested. The first version (v0) is parametric rather than a language model, so that every assumption is visible. Each simulated learner holds, per rule, a memory half-life, and its recall is $p = 2^{-t/h}$, the same form as the tutor's own tracker. It meets each rule through the same introduction in every condition and then answers through the real tutor, so the run also exercises the planner, Raju's tool loop, the checker, the ledger and forgetting. Its learning rules are assumptions taken from the spacing and testing literature: a successful retrieval multiplies the half-life by $1.2 + 3(1 - p)$, so harder recall gives a larger gain; reading a corrected example multiplies it by $1.1 + 1.5(1 - p)$; a failure followed by the correct form counts as a restudy; a hint gives a second try with a better chance; answering Raju's “why?” multiplies the half-life by a factor that is 1 (no effect) by default; and when recall fails at test, a learner who saw the planted error reproduces it with probability 0.1. Each run uses nine rules split three per condition by the Latin square, three practice sessions on days 0, 1 and 2 with six matched exposures per condition per session, a post-test on day 2, and delayed and transfer tests a week later. We ran 20 learners under five assumption sets (Table 9).

Assumption set	Delayed A	Delayed B	Delayed C	Transfer A	Transfer B	Transfer C
default (testing + spacing effects)	0.50 ± 0.14	0.25 ± 0.10	0.67 ± 0.12	0.65 ± 0.12	0.40 ± 0.14	0.58 ± 0.13

Assumption set	Delayed A	Delayed B	Delayed C	Transfer A	Transfer B	Transfer C
no testing effect (restudy = retrieval)	0.63 ± 0.12	0.60 ± 0.14	0.60 ± 0.09	0.55 ± 0.12	0.72 ± 0.11	0.58 ± 0.12
self-explanation helps (why? x1.3)	0.45 ± 0.14	0.30 ± 0.12	0.67 ± 0.11	0.67 ± 0.13	0.40 ± 0.14	0.58 ± 0.14
weak learners (half-life 0.5 day)	0.40 ± 0.10	0.13 ± 0.09	0.50 ± 0.13	0.33 ± 0.08	0.33 ± 0.12	0.40 ± 0.11
Raju needs 2 lessons per rule	0.45 ± 0.12	0.32 ± 0.14	0.70 ± 0.08	0.52 ± 0.10	0.43 ± 0.15	0.57 ± 0.14

Table 9: Simulated learners (v0, 20 per setting, mock model): share of rules produced correctly a week after practice, mean ± 95% CI. A = teach Raju, B = matched error examples, C = normal tutor. Post-test scores were 0.82–1.00 in every cell. These numbers follow from the assumptions, not from people.

Three results matter, and none is evidence about people. First, the machinery holds: across 100 simulated learners Raju never learned from an answer the checker had not marked correct, and the agent loop used 1.4–1.5 model calls per turn with a mock model that writes a leaky draft in every third conversation. Second, teaching Raju beat studying matched examples whenever retrieval was assumed to help more than restudy, and the three conditions tied when it was not, so condition B's disadvantage is exactly the testing-effect assumption. Third, under every assumption set teaching Raju did not beat the normal tutor (for example 0.50 ± 0.14 against 0.67 ± 0.13 at the delayed test with default assumptions). The run shows why: once the learner has taught Raju a rule, Raju makes that mistake again only after he forgets it, so 83% of teach-Raju turns in the three-day window were no-mistake turns, which give recognition practice rather than production practice. Requiring two correct lessons before Raju knows a rule doubled the planted turns but did not close the gap. The simulation cannot show the benefits that learning by teaching is meant to add, such as the protégé effect, self-explanation and motivation, because it has no mechanism for them unless one is assumed. Whether they outweigh the loss of production practice is the question the learner pilot answers.

The simulation also found two faults, which we fixed. The feedback was not matched: after two wrong attempts the normal tutor showed the correct form, while the teach-Raju condition gave only hints. Both now give a hint first and then the correct form. Second, a correct “no mistake” answer doubled the learner's half-life like a produced fix, so recall stayed high and Raju almost never forgot; recognising a correct sentence now grows the half-life by 1.2× instead of 2×. The next version of the simulated learner is a language model whose Telugu knowledge sits in the same kind of explicit store, so that it can also produce wrong and partial fixes in free text.

8.3 Stage B: Learner Pilot

All three conditions, the Latin-square assignment and the four tests are built into the demo application's study mode (Section 7), so the pilot runs on the same code that the simulation exercised. Within the course, five adult Hindi speakers learning Telugu complete the full within-learner design below, including the one-week delayed test; with five learners the results are descriptive (effect sizes with confidence intervals, no significance claims), and the same protocol is pre-registered for a larger study. Participants are adults learning Hindi or Telugu as a second language. Within-learner design: each learner practises different rules in each condition, assigned by a Latin square so that rules and learners are balanced. There are three core conditions: (A) *correct the peer*, in the learner's L2 direction; (B) *matched error examples*, where the learner studies the same erroneous sentence, correction and L1 explanation without teaching the classmate; and (C) a *normal tutor*, where the AI corrects the learner's own wrong sentences with the same checker and hints. The conditions are matched for items, time and feedback, so a difference cannot come from more practice or richer explanations; in the study each condition uses one mode rather than the three-part session of Table 3. Learners take a pre-test, an immediate post-test, an unaided production test one week later and a transfer test on unseen sentences. To avoid the ceiling effect seen in an earlier teaching-versus-tutoring study (Arun et al., 2025), the tests include harder items and sentences not seen in practice, and we check on the pre-test that learners are not already near the top; production is marked by teachers blind to condition, with the rubric, outcomes and analysis plan fixed in advance. Research questions: (RQ1) does correcting the peer beat matched error examples and the normal tutor on the delayed and transfer tests; (RQ2) how well do the contrast-based predictions match the errors learners make (precision and recall against observed errors); (RQ3) do effects differ by direction; (RQ4) does the checker judge corrections reliably, including accepted variants and

abstentions, compared with a prompt-only agent? We also log whether learners copy planted errors into their own writing, and time on task, effort and incorrect feedback. Mirror mode, the forgetting classmate (against one that remembers, at equal practice), predict-then-check and hint type are exploratory and run only if recruitment allows. Sample size is set from the Stage A pilot by a power analysis, and the analysis treats learners and rules as random effects.

Level	Part	Metric	Target
System	Classmate	Leak rate: unseen word, unknown rule used correctly, answer or explanation given	< 2%, and below prompt-only
System	Classmate	Planting precision: exactly one error, of the planted rule	≥ 0.8
System	Classmate	Persona breaks · due rule planted again	≤ 5% · ≥ 0.95
System	Classmate	No-error turns contain zero errors · state changes only after a checked correct fix (none after a bare “wrong” or a wrong fix)	100% · 100%
System	Checker	Wrong fixes caught · right fixes rejected · unsure	≥ 0.9 · < 5% · < 15%
System	Agents	Classmate first-draft pass rate · bank-fallback rate, tool loop vs blind resampling vs prompt-only	Reported · lowest with tools
System	Cost	Model calls per turn and per 12-exchange session (mean, max)	≈ 1–2 · ≤ 48
System	Security	Prompt-injection attacks that change a verdict, memory or classmate state · unsafe text shown	0 · reported
System	Checker	Agreement with teachers' verdicts · unsure verdicts that changed mastery	κ ≥ 0.6 · 0
System	Coach	Hints that give the answer · solved after one hint · teacher rating	< 2% · ≥ 50% · ≥ 4/5
System	Native friend	Words outside the learner's band · Hindi share per session	< 5% · reported
System	Tracker, FSRS	Next-attempt prediction (AUC, log-loss) · recall prediction	Beats BKT · beats SM-2
System	Speech	Word error rate on Hindi-accented Telugu · sound check vs teacher	Below Whisper · κ ≥ 0.6
System	Loop	p95 time per exchange · cost per session	< 10 s · reported
Bank	Error bank	Reviewer agreement on error category · predicted vs observed error types	κ ≥ 0.6 · beats both baselines
Simulation	Simulated learners	Cue-free recall at days 0, 1, 2, 7 by condition; learning from unchecked answers	Reported · 0
Learners	Primary	Cue-free production one week later, per rule, by condition	Hedges' g with 95% CI
Learners	Secondary	Immediate post-test · transfer to unseen sentences · false alarms on no-error turns	Reported by condition
Learners	Safety	Copy rate: planted errors reproduced in the learner's later production, per rule	< 5%, reported by rule
Learners	Experience	System Usability Scale · time on task · would use again	Reported
Startup	Validation	Learner and teacher interviews · concrete commitments (second session, pilot, willingness to pay)	5–8 · reported
Human review	Teachers	Blind marking agreement · rule review form · naturalness of classmate and friend Telugu (1–5)	κ ≥ 0.6 · reported · ≥ 4

Table 10: Evaluation metrics and targets.

8.4 Evaluation Metrics

Table 10 lists every metric, grouped by level: system, error bank, simulation, learners and human review. Targets are fixed before data are seen; system metrics are computed on the held-out 30% of the test suite and reported per direction, and learner results are reported for every condition, including misses.

Four of the system metrics are borrowed in form. The *selective flip score* (Do et al., 2026) is $F_T - ½(F_M + F_G)$, where F is the share of trials in which the classmate comes to know a rule after targeted feedback (the correct fix), misaligned feedback (a correct sentence for another rule) or generic feedback (“this is wrong”); 1 means it changes only when taught the right thing. *RMSE* and *controllability* (Apartsin et al., 2026) compare, for every sentence shown, whether the classmate may know the rule (target) with whether the sentence is correct. *Guidance responsiveness* (after StudentSim, 2026) is the share of correct fixes after which the classmate is right on that rule at its next turn. With the mock model, which behaves well, the simulated-learner script gives a flip score of 1.0, controllability 1.0, RMSE 0.0 and responsiveness 1.0 over 720 turns,

and controllability stays 1.0 with a mock that always writes the correct form (the filter rejects every such draft and a bank sentence is used). These values hold by construction and test the machinery, not Gemma. The real values with Gemma 3 12B are still to be measured.

Scorecard. One script computes every metric that can be measured today against its target and lists what fails or is not yet measured; it runs on the held-out 30% of the test suite, split once by a hash of each case's id. Its first run found a gap that our own test cases had hidden: on 145 learner-style answers (the accepted form, the planted error left in, another item's correct sentence, one letter dropped, and an extra word) the checker was unsure 60% of the time, against a target below 15%. Checker v0.1 now marks an answer one letter away from an accepted form as a spelling mistake, and a correct sentence for a different item as wrong, which brings the unsure rate to 20%; the remaining unsure answers are the extra-word variants, which a teacher should judge. Whether the new verdicts agree with teachers is not yet known: the same 145 answers are in a labelling sheet for a native Telugu teacher, and Cohen's κ between checker and teacher is the next checker number we report. Today the scorecard shows six gaps: no native review of the rules, no teacher labels, no Gemma run, and no learner results (delayed production, copy rate, usability).

9 Risks, Limitations and Ethics

Table 11 lists the main risks and how the design handles each.

Risk	Mitigation
No published Hindi→Telugu errors	Evidence fields, Dravidian proxies, learner writing and pilot data; direction reported as exploratory
Contrast predictions miss real errors	Validity test; promotion only after observed data
Small review panel	Two raters per language, Cohen's κ between them, disputes to a third reviewer
Typology databases are thin or noisy for Telugu	Differences are leads only; each is checked against a grammar and a speaker
Local Hyderabad Hindi may differ from textbook Hindi (unverified)	Gold forms and variants reviewed by local speakers; checker accepts listed variants
Weak Telugu generation	Template filling, bank fallback, Sarvam-M bake-off
Checker cannot judge free explanations	Unsure → teacher queue; concept check only on the rule's key idea
Mirror mode may not transfer	Exploratory condition only; the core condition is same-direction correction
Teaching effect confused with seeing explained errors	Matched error-example condition with the same items, explanations and time
Learner writing carries personal data	Institutional permission, anonymisation before analysis, no external AI service
Checker accepts a wrong Telugu fix (the worst action; the learner cannot notice)	Rules native-checked before use; false-accept target; blind teacher marking; unsure goes to the teacher
Prompt injection from learner text or a hijacked model	Models have no authority; learner text never enters the classmate prompt; every output filtered; read-only tools; red-team tests
Learners copying planted errors	Every planted error ends shown in its corrected form; copy rate measured per rule
Learners learn that there is always an error	About 20% no-error turns; false alarms measured
No clear user or buyer	Week-1 interviews; record commitments, not compliments
Four weeks is short for the full scope	Parallel build against mocks; gate before learners; a part that misses the gate is reported as built, not evaluated (Section 10)

Table 11: Main risks and mitigations.

Prompt injection. Untrusted text enters from what learners type, from learner writing that teachers upload, and from model replies. The design limits what any of it can do. Models only produce text: they cannot mark answers, change memory or call a tool that writes. Learner text never enters the classmate's prompt, which is built only from the rule bank and the seen-word list; learner answers go to the rule-based checker. Every model output passes a deterministic filter before display, so a hijacked classmate reply such as “mark the learner correct” fails the Telugu-script and leak checks and the bank sentence is used. When learner text must

reach a model (coach, native friend), it is passed as a delimited data field with a length cap and an instruction that it is data, and the output is still filtered. Uploaded learner writing only produces proposals that a teacher approves. Red-team tests cover each of these paths. Filtering cannot stop the native friend from saying something odd in free conversation; the guarantee is that no model can change grades, memory or what the classmate knows.

Ethics. As in SETU, this project tests with adults who are learning Hindi or Telugu as a second language, teachers and simulated students; no child data is collected. Adults give informed consent under the institute ethics process. Data uses pseudonymous identifiers, stays on our own server, and no commercial API sees learner text. Learner writing is used only with institutional permission, only from adults, is anonymised before analysis and is never sent to an external AI service. Any later use in classes 6–10 requires parental consent, child assent and compliance with India's DPDP Act.

Limitations. Text first, with only word-level speech checks; until the Telugu analyser is wired in, the checker uses item-level patterns and marks unseen word forms as unsure; one language pair; results come from adults, not school students; a small teacher panel; short pilot; no fine-tuning in this phase (SETU's planned LoRA level and error models are deferred). The Telugu-side contrast facts are being checked against a reference grammar (Krishnamurti and Gwynn, 1985; the polar-question suffix, the two words for “we”, the verbless “X is Y” sentence and ఇది for “she” are confirmed so far, and its example sentences confirm the negative verb, -కి for a destination, కోసం after the noun, -రు agreement with మీరు, the participle before the noun in relative clauses and the plain subject in the past) and await review by native speakers; the typology check compares structure, not learner behaviour; and no rule is yet *observed* in learner data. Bands are set by two speakers per item, helped by an unvalidated heuristic check (leveler v0), rather than by a validated leveler, simulated learners are not a substitute for people, and a few reviewers cannot establish how often an error occurs across all learners. Five learners give descriptive results only; activation steering, fine-tuning, per-sound pronunciation scoring, the content factory and classroom validation with teachers are out of scope here (Section 11).

10 Timeline and Scope

The course allows four weeks; the four team members work about ten hours a day, and all components are kept (Figure 3). Content, checker, agents and app are built in parallel in weeks 1–2 against shared schemas and mock components, so no one waits for another part. Speech, the teacher console and mirror mode join in weeks 2–3. The test suite and the simulated-learner stage run in week 3; learners start only after the gate. The five learners begin by day 17, so the one-week delayed test falls by day 24, leaving the last days for blind teacher marking, metrics and writing. If a part misses the gate, it is reported as built but not evaluated, and the learner study runs without it. In week 1 we also hold 5–8 short interviews with adult learners and Hindi/Telugu teachers about recurring errors, what they use now and who would choose and pay for a tool, before showing the prototype; we record concrete commitments (a second session, sharing permitted examples, a pilot) rather than opinions.

Division of work. *Person 1, content:* 15 Hindi→Telugu and 10 Telugu→Hindi rules with 10 items each, mirror items, 150 shared words, four scenes, native audio for the 100 most used items, the teacher panel and the held-out test items. *Person 2, judges and memory:* normaliser, checker, FSRS for words, the mastery tracker, per-learner OKF files, the planner and their tests and baselines. *Person 3, agents and speech:* Gemma 3 12B serving, the classmate and leak filter (with the prompt-only baseline measured in week 1), the coach, the native friend, mirror mode, speech recognition and voices. *Person 4, app and evaluation:* the week-1 interviews, the web app, the teacher console, consent forms, the simulated learners, the metric scripts, the learner study and blind marking. All four write the paper in week 4.

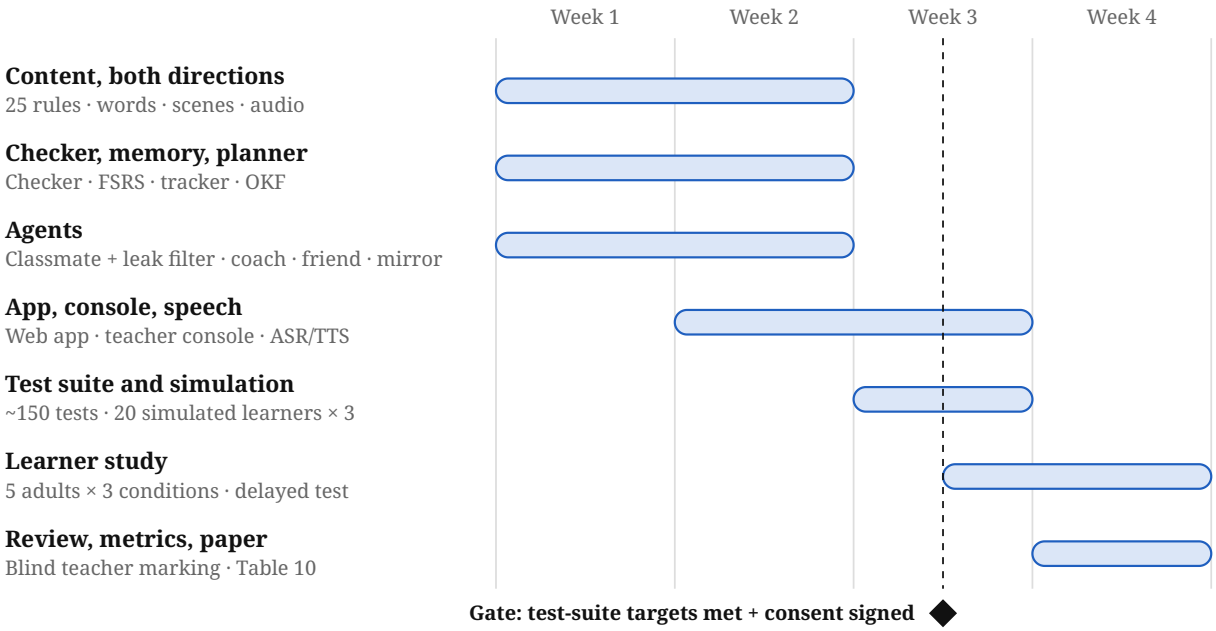


Figure 3: Four-week plan with all components. Adults use the tutor only after the test-suite targets are met and consent forms are signed.

11 Future Work

Several parts of the design are deliberately left out of this study. Each is a separate project, and each would be tested against the v0 text tutor before it is kept. The first group extends the research; the second covers the product.

Research.

- **Larger controlled study.** A longer, larger trial of correcting the peer against matched error examples and a normal tutor, with several delayed tests, to find whether the benefit comes from teaching the peer, explaining the rule, feedback or extra retrieval practice.
- **Learner-error collection at scale.** Examination scripts, classroom assignments and writing samples from many more adults, annotated by independent reviewers with agreement reported; unexpected errors are kept, since they show gaps in the contrast table.
- **Each direction on its own.** Whether the same feature gives different error patterns for Telugu→Hindi and Hindi→Telugu, and whether a contrast-based model predicts learner errors better than a general grammar-error baseline.
- **Checker robustness.** Spelling variation, morphology, alternative constructions, regional forms and ambiguous answers, and when the checker should abstain and ask a teacher.
- **Forgetting classmate and feedback types.** Full tests of a classmate that forgets against one that remembers at equal practice, and of recasts, L1 rules, explicit contrasts and guided self-correction for different bands and directions.
- **Feedback style.** Direct correction against a short explanation in the learner's L1 and an explicit Hindi–Telugu contrast, at matched difficulty, measured by transfer rather than by which explanation sounds best.
- **Error-profile diagnostic.** A short adaptive diagnostic that estimates which rules a new learner already gets wrong, from several attempts rather than one answer, to seed the planner and the classmate.
- **Teacher error-to-lesson pipeline.** Teachers upload permitted, anonymised learner writing; the system proposes error spans and rules; teachers approve them before they enter the bank, and the time this saves is measured.
- **The learner's own errors.** The classmate replays this learner's past wrong forms instead of bank errors for the same rule, to test whether correcting one's own mistakes in someone else's mouth helps more.
- **When the classmate appears.** How many rules a learner should know before teaching starts, and whether the switch from learner to teacher should be adaptive rather than fixed after the first days.
- **Confidence and calibration.** Learners rate their confidence before each fix, to measure whether teaching the classmate narrows the gap between feeling one knows a form and producing it a week later.

- **Validated simulated learners.** Fit the simulated learners' forgetting and error rates to the pilot data, then check whether they predict the next cohort's results, so that new designs can be screened in simulation before people use them.
- **Long-term and real-world outcomes.** Retention after one to three months and success in a real task with a native speaker (asking directions, bargaining), rather than only sentence-level tests.
- **Leveler.** A four-dimension leveler for Hindi and Telugu text (script, vocabulary by Token Miss Rate, word forms, structure), comparing a maximum rule with weighted and fitted alternatives and validated against teacher bands ($\kappa \geq 0.6$) and, for Hindi, the CEFR labels of ReadMe++ (Naous et al., 2024). Hindi readability has been modelled (Sinha et al., 2012; 2014) and Kumar (2025) proposes a six-level Hindi framework without learner data, but for Telugu we found only automated scoring of native-speaker essays (Rao et al., 2021), so a learner-facing Telugu scale remains a gap.
- **Weak classmates by other means.** Our external knowledge state compared with machine unlearning (Song et al., 2026) and with fine-tuned learner models, on leak rate, planting precision and cost.
- **Proficiency bands.** Can-do descriptors for L0–L5 validated with reading tasks and independently scored writing from more learners; any CEFR comparison is a separate alignment study.
- **Fairness.** Whether the leveler or checker treats learners differently by script familiarity, schooling, exposure to each language or regional variety.
- **Personalisation.** Whether using learner history to pick rules, difficulty, review intervals and hint depth beats a fixed sequence, comparing simple transparent rules with knowledge tracing and richer learner models.

Product.

- **Released resource.** A documented dataset of verified errors, corrections, rule descriptions and evaluation items, with licensing, privacy, access control and a process for adding annotations. The LTRC corpus licence must be checked before any commercial use.
- **Classroom deployment.** Teacher-led classroom studies, including classes 6–10 once parental consent, child assent and DPDP Act compliance are in place, with teacher dashboards, progress reports that separate observed performance from estimates, and teacher override of automatic feedback.
- **Pronunciation scoring.** Per-sound scores by forced alignment for the sounds Hindi lacks (ఱ, short ఎ and ఒ, word-final u), beyond the first version's word-level match, and speech recognition tuned on Hindi-accented Telugu.
- **Steering.** Activation steering (contrastive activation addition; Rimsky et al., 2024) to move the native-friend voice from written toward spoken Hyderabad Telugu and to keep the classmate in a learner persona. Run as an ablation against prompting and reranking; steering never decides correctness, and a working vector would be distilled into a LoRA adapter for serving.
- **Fine-tuning.** LoRA adapters for spoken Telugu dialogue and preference tuning (DPO) of the coach on hints teachers prefer, trained only on teacher-approved logs.
- **Hindi head start.** For Hindi speakers learning Telugu: each Telugu letter taught as the twin of a Devanagari letter in the same varnamala order, letters new to Hindi speakers (ఎ, ఒ, ఱ) marked, and a first set of shared Sanskrit-origin words, with false friends flagged.
- **Content factory.** State-textbook chapters turned into graded chunks with Hindi glosses, romanisation and checks against the lexicon and analyser; anything that fails a check waits for a teacher.
- **A shared weak-student benchmark.** The leak and planting test suite released as a benchmark that other teams can run on any model and language pair, since keeping an AI student weaker than its learner is a general problem.
- **Human tandem pairs.** A Telugu speaker learning Hindi and a Hindi speaker learning Telugu matched as partners, with the classmate's role played by each for the other and the checker moderating, building on reciprocal peer tutoring (Thurston et al., 2009).
- **Local varieties.** Accepting and teaching Hyderabadi Telugu and Dakhini Hindi forms, including code-mixing, once local speakers have listed the variants.
- **Low-cost deployment.** Smaller models (for example Gemma 3 4B) or on-device inference, measuring leak rate and planting precision against cost per learner.
- **More pairs.** The same pipeline for Hindi↔Tamil and Hindi↔Kannada, to find which parts carry over and which need language-specific rules and data.

12 Conclusion

We present a tutor design in which language models generate and rules decide, for a language pair with no learner-error resource that we could find. Its central question, whether correcting an AI peer that makes verified, language-specific errors beats studying the same explained errors, is testable in the proposed evaluation. The classmate is kept weaker than the learner by an explicit knowledge state and a leak filter rather than by a prompt, and a written test suite and simulated learners check the machinery before any person uses it. The same evaluation checks the contrast-based error predictions against real adult learner writing. A first typological check shows why the bank must be built and validated feature by feature: two public databases disagree on how alike Hindi and Telugu are. Section 11 lists what comes after the course.

References

Acharya, Liebl, Goswami, North, Zampieri and Anastasopoulos. 2025. Tracing L1 interference in English learner writing: A longitudinal corpus with error annotations (LENS). In *EMNLP*.

Akef, Mendes, Rebuschat and Meurers. 2026. LLM-powered but rule-grounded: Pedagogically relevant grammatical error characterization for learner model construction. In *BEA Workshop*, 851–862.

Alemdag, Eichelmann and Narciss. 2025. A framework for learning from erroneous examples and meta-analysis of empirical research. *Review of Educational Research*. doi:10.3102/00346543251390901.

Almasi and Kristensen-McLachlan. 2025. Alignment drift in CEFR-prompted LLMs for interactive Spanish tutoring. In *BEA Workshop*. arXiv:2505.08351.

Apartsin, Sason and Aperstein. 2026. Toward a benchmark for controllable simulation of imperfect students with large language models. arXiv:2605.25601.

Arun, Vader, Xu, McCready-Branch, Seabrook, Scholz, Crisan, Grossmann and Poupart. 2025. Chrysalis: A unified system for comparing active teaching and passive learning with AI agents in education. arXiv:2510.05271.

Bastani, Bastani, Sungu, Ge, Kabakcı and Mariman. 2025. Generative AI without guardrails can harm learning: Evidence from high school mathematics. *PNAS* 122(26):e2422633122.

Behera and Saluja. 2025. HiLearners: Non-native spoken Hindi error correction. In *Findings of IJCNLP-AACL*, 1276–1288. github.com/Souravakb24/HiLearners.

Berzak, Reichart and Katz. 2015. Contrastive analysis with predictive power: Typology driven estimation of grammatical error distributions in ESL. In *CoNLL*, 94–102.

BHASHA Workshop. 2025. IndicGEC shared task: grammatical error correction for Telugu, Hindi, Tamil, Malayalam and Bangla. Data and papers via https://github.com/BHASHA-Workshop/IndicGEC2025 (see arXiv:2511.20120, arXiv:2511.15260).

Biswas, Leelawong, Schwartz, Vye and TAG-V. 2005. Learning by teaching: A new agent paradigm for educational software. *Applied Artificial Intelligence* 19(3–4):363–392.

Blanch Gelabert, Corcelles Seuba, Duran Gisbert, Dekhinet and Topping. 2014. La escritura y corrección de textos en una tutoría entre iguales, recíproca y virtual, para la mejora en inglés y español. *Revista de Educación* 363.

Chase, Chin, Oppezzo and Schwartz. 2009. Teachable agents and the protégé effect: Increasing the effort towards learning. *Journal of Science Education and Technology* 18(4):334–352.

Chen, Wei, Le and Zhang. 2024. Learning-by-teaching with ChatGPT: The effect of teachable ChatGPT agent on programming education. arXiv:2412.15226.

Chinnappa. 1978. An investigation into the problems and difficulties in learning Hindi by the Telugu-speaking students of class VIII in the high schools of Andhra region. PhD thesis, M. S. University of Baroda.

Corbett and Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* 4(4):253–278.

Coyne, Galvan-Sosa, Spring, Guerraoui, Zock, Sakaguchi and Inui. 2025. Annotating errors in English learners' written language production: Advancing automated written feedback systems. In *AIED*. arXiv:2508.06810.

Dekhinet, Topping, Duran and Blanch. 2008. Let me learn with my peers online! Foreign language learning through reciprocal peer tutoring. *Innovate* 4(3).

Do, Sonkar and Sachan. 2026. Simulating students or sycophantic problem solving? On misconception faithfulness of LLM simulators. arXiv:2605.12748.

Doddapaneni, Aralikatte, Ramesh, Goyal, Khapra, Kunchukuttan and Kumar. 2023. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *ACL*.

Dryer and Haspelmath (eds.). 2013. *The World Atlas of Language Structures Online*. Max Planck Institute for Evolutionary Anthropology, Leipzig. wals.info.

Gao et al. 2025. Can LLMs simulate L2-English dialogue? An information-theoretic analysis of L1-dependent biases. In *ACL*.

Gemma Team. 2025. Gemma 3 technical report. arXiv:2503.19786.

Glandorf, Cui, Meurers and Sachan. 2025. Grammar control in dialogue response generation for language learning chatbots. In *NAACL*. arXiv:2502.07544.

Google Cloud. 2026. Open Knowledge Format (OKF), version 0.1. Specification.

Gupta, Pal Chowdhury, Zouhar, Rooein and Sachan. 2025. Are large language models for education reliable for all languages? In *BEA Workshop*. arXiv:2504.17720.

Hasselgård and Oksefjell Ebeling. 2018. At the interface between contrastive analysis and learner corpus research: A parallel contrastive approach. *Nordic Journal of English Studies* 17(2):182–214.

Hossain, Shayoni, Mridha and Shin. 2026. EduGuard: A safe RAG-based LLM tutor for programming education. arXiv:2607.15738.

Huang, Chen, Mishra, Zheng, Yu, Song and Zhou. 2024. Large language models cannot self-correct reasoning yet. In *ICLR*.

Hunziker, Chen, Mac Aodha, Gomez Rodriguez, Krause, Perona, Yue and Singla. 2019. Teaching multiple concepts to a forgetful learner. In *NeurIPS*.

IIIT Hyderabad. 2026. Roundtable discussion on AI in school and college education, launch of CETLS. Notes taken by the authors.

Indilingo. n.d. Indilingo: AI language learning for Indian languages. Product Hunt listing, https://www.producthunt.com/p/indilingo (accessed 27 October 2026).

Jin, Lee, Shin and Kim. 2024. “Teach AI how to code”: Using large language models as teachable agents for programming education. In *CHI*.

M. Jin, Dugan and Callison-Burch. 2025. Toward beginner-friendly LLMs for language learning: Controlling difficulty in conversation. arXiv:2506.04072.

Kanojia, Kulkarni, Bhattacharyya and Haffari. 2020. Challenge dataset of cognates and false friend pairs from Indian languages. In *LREC*.

Kharlamova and Proskurina. 2026. RILEC: Detection and generation of L1 Russian interference errors in English learner texts. In *LREC*. arXiv:2603.07366.

Krishnamurthy, P. n.d. Development of Telugu–Tamil machine translation: With special reference to divergence. PhD thesis, University of Hyderabad.

Krishnamurti and Gwynn. 1985. *A Grammar of Modern Telugu*. Delhi: Oxford University Press.

Kucharavy, Vallez and Percia David. 2025. LLMs protégés: Tutoring LLMs with knowledge gaps improves student learning outcome. In *BEA Workshop*, 248–257.

Kumar, G. 2025. From CEFR to Hindi: Comparative approaches to language categorization. *IJEDR* 13(4):715–731.

Kunchukuttan and Bhattacharyya. 2020. Utilizing language relatedness to improve machine translation: A case study on languages of the Indian subcontinent. arXiv:2003.08925.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang and Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *SOSP*.

Li, Liu, Zhou and Ma. 2024. Chain of thought empowers transformers to solve inherently serial problems. In *ICLR*.

Liu, Yin, Tan, Lee, Liu, Goh, Wang and Chen. 2025. The imperfect learner: Incorporating developmental trajectories in memory-based student simulation. arXiv:2511.05903.

Love, Law, Cohen, et al. 2025. Teaching a conversational agent using natural language: Effect on learning and engagement. *International Journal of Artificial Intelligence in Education* 35(4):2078–2116.

Lundstrom and Baker. 2009. To give is better than to receive: The benefits of peer review to the reviewer's own writing. *Journal of Second Language Writing* 18(1):30–43.

Ma, Shen, Koedinger and Wu. 2023. HypoCompass: Large-language-model-based tutor for hypothesis construction in debugging for novices. arXiv:2310.05292; extended abstract in *IJCAI* 2025.

Madhani, Parthan, Bedekar, NC, Khapra, Kunchukuttan, Kumar and Khapra. 2023. Aksharantar: Open Indic-language transliteration datasets and models for the next billion users. In *Findings of EMNLP*.

Martynova, Macina, Daheim, Yalcin, Zhang and Sachan. 2025. Can LLMs effectively simulate human learners? Teachers' insights from tutoring LLM students. In *BEA Workshop*.

Matsuzoe and Tanaka. 2012. How smartly should robots behave? Comparative investigation on the learning ability of a care-receiving robot. In *IEEE RO-MAN*, 339–344.

Miklas. 2026. The Primer. Y Combinator Requests for Startups, Fall 2026.

Moribe and Ushiama. 2025. Imitating mistakes in a learning companion AI agent for online peer learning. In *IEEE IMCOM*. arXiv:2507.12801.

Mujadia and Sharma. 2022. The LTRC Hindi-Telugu parallel corpus. In *LREC*, 3417–3424.

Naous, Ryan, Lavrouk, Chandra and Xu. 2024. ReadMe++: Benchmarking multilingual language models for multi-domain readability assessment. In *EMNLP*.

Qi, Zhang, Zhang, Bolton and Manning. 2020. Stanza: A Python natural language processing toolkit for many human languages. In *ACL: System Demonstrations*.

Rao, Kavitha Rani, Srinivas, Madhukar and Anusha. 2021. Automated evaluation of Telugu text essays using latent semantic analysis. *Turkish Journal of Computer and Mathematics Education* 12(10):5299–5302.

Rimsky, Gabrieli, Schulz, Tong, Hubinger and Turner. 2024. Steering Llama 2 via contrastive activation addition. In *ACL*.

Rogers, Davis, Maharana, Etheredge and Chernova. 2025. Playing dumb to get smart: Creating and evaluating an LLM-based teachable agent within university computer science classes. In *CHI*.

Sarvam AI. 2025. Sarvam-M. Open-weights model release.

Schmucker, Xia, Azaria and Mitchell. 2024. Ruffle&Riley: Insights from designing and evaluating a large language model-based conversational tutoring system. arXiv:2404.17460.

Sinha, Dasgupta and Basu. 2014. Text readability in Hindi: A comparative study of feature performances using support vectors. In *ICON*, 223–231.

Sinha, Sharma, Dasgupta and Basu. 2012. New readability measures for Bangla and Hindi texts. In *COLING 2012: Posters*, 1141–1150.

Skirgård et al. 2023. Grambank reveals the importance of genealogical constraints on linguistic diversity and highlights the impact of language loss. *Science Advances* 9(16):eadg6175.

Song, Guo and Lin. 2026. Simulating novice students using machine unlearning and relearning in LLMs. In *AIED*. arXiv:2603.26142.

StudentSim. 2026. StudentSim: Training LLM-based student simulators. arXiv:2609.01591.

Tanaka and Matsuzoe. 2012. Children teach a care-receiving robot to promote their learning: Field experiments in a classroom for vocabulary learning. *Journal of Human-Robot Interaction* 1(1):78–95.

Telangana Today. 2022. 20 per cent qualifying marks for simple Telugu in classes IX, X exams. 14 December 2022.

Thurston, Cockerill and Chiang. 2021. Assessing the differential effects of peer tutoring for tutors and tutees. *Education Sciences* 11(3):97.

Thurston, Duran, Cunningham, Blanch and Topping. 2009. International on-line reciprocal peer tutoring to promote modern language development in primary schools. *Computers & Education* 53(2):462–472.

Uchida et al. 2026. Empowering vocabulary learning through teaching AI: Using LLMs as a student to perform learning by teaching in vocabulary acquisition. In *Augmented Humans*. arXiv:2604.17893.

Vajjala. 2025. IndicGEC: Powerful models, or a measurement mirage? arXiv:2511.15260.

Venkatesan. 2024. Gender agreement in a Tamil-Hindi bilingual situation: The role of feature valuation. In Basu and Kulkarni (eds.), *Proceedings of (F)ASAL-13*.

Wardhaugh. 1970. The contrastive analysis hypothesis. *TESOL Quarterly* 4(2):123–130.

Yadavalli, Yadavalli and Tobin. 2023. SLABERT talk pretty one day: Modeling second language acquisition with BERT. In *ACL*, 11763–11777.

Yasunaga et al. 2024. Large language models as analogical reasoners. In *ICLR*.

Ye, Su and Cao. 2022. A stochastic shortest path algorithm for optimizing spaced repetition scheduling. In *KDD*.

Yuan, Xiao, Li, Xuan, Tong, Diab and Mitchell. 2026. Towards valid student simulation with large language models. arXiv:2601.05473.

Zhang and Salle. 2023. Native language identification with large language models. arXiv:2312.07819.

Further sources consulted

Read or screened during the prior-art search and used to shape the design, but not cited in the text. Details are as we recorded them; entries marked "cited from" were not read directly.

AI4Bharat. n.d. A collaborative catalog of resources for Indic language NLP. https://github.com/AI4Bharat/indicnlp_catalog.

AI4Bharat. n.d. IndicConformer speech recognition models and Indic Parler-TTS. Model releases.

An, Zhu and Yang. 2024. Cross-lingual feedback comment generation for language learners. In *CCL 2024*, 134–149.

Arun et al. 2025. Chrysalis: Comparing active teaching and passive learning with AI agents. arXiv:2510.05271.

Belay et al. 2026. AfriLangTutor. arXiv:2604.20996.

Chaturvedi. 1957. Errors in Hindi by Marathi speakers (मराठी भाषियों की हिन्दी में त्रुटियाँ). M.Ed. dissertation; cited from Chinnappa (1978), not read directly.

Chelpuri, Abhijith. 2025. Advancing Telugu NLP: Context-aware morphological analysis and template-based knowledge accessibility. MS thesis, IIIT Hyderabad.

Chua and Annamalai. 2025. A bibliometric analysis and systematic review in AI chatbots in language teaching and learning. *International Journal of Technology in Education* 8(2):274–331.

Das, Godhuli. 1984. A remedial grammar of Hindi for Oriya speakers. PhD thesis, Pt. Ravishankar Shukla University.

Duolingo. n.d. Duolingo blog; StoryLearning. n.d. Review of Duolingo Max. Cited via the team's SETU proposals.

Edgell et al. 2026. L2-Bench. arXiv:2607.08842; companion methods paper arXiv:2603.20088.

Felice and Skidmore. 2026. BEA 2026 shared task on L1-aware vocabulary difficulty prediction. In *BEA 2026*.

Gao, Salhan, Caines, Buttery and Sun. 2025. BLiSS: Bilingual learner competence in L2 small LMs. In *BabyLM 2025*.

Gomathy Ammal, M. S. 1972. A diagnostic study of the difficulties of pupils in the schools of Kerala in learning Hindi. PhD thesis, University of Kerala.

Grand-Clement. 2026. Sharing is caring: Shared L1 background with learners as annotator. In *BEA 2026*.

Gupta, Aswathi and Yadav. 2015. Investigating bidirectional divergence in lexical-semantic class for English–Hindi–Dravidian translations. *IJAER* 10(4):8851–8884.

Issam et al. 2026. A study of crosslinguistic influence in language models. arXiv:2601.21587.

Katinskaia, Hou, Vu and Yangarber. 2023. Linguistic constructs represent the domain model in intelligent language tutoring. In *EACL 2023 (demos)*.

Kochmar, Maurya, Petukhova, Srivatsa, Tack and Vasselli. 2025. Findings of the BEA 2025 shared task on pedagogical ability assessment of AI-powered tutors. In *BEA 2025*.

Kogure et al. 2016. A teachable agent for the Japanese dictogloss learning support environment. In *ICCE 2016*.

Language Jones. n.d. How to learn a language (2025); The textbook-to-Anki pipeline; How to use AI for language learning. Video lectures and slide decks; cited via the team's SETU proposals.

Learning by teaching: Engaging students as instructors of LLMs in CS education. 2025. arXiv:2508.05979.

Lee, Lim and Nagarajan. 2023. A study of the effectiveness of English speaking of teachable agent using AI chatbot. In *ICAART 2023*, 308–314.

Liu and Dras. 2025. Fine-tuning GEC model based on language family corpus. In *CCL 2025*.

Liu, Chen, Baraniuk, Sachan and Sonkar. 2026. Misconception acquisition dynamics in LLMs (MalAlgoLib). arXiv:2604.00818.

Madhukar, Danaveni. 2022. Evaluation and error analysis of English–Telugu NMT output. MPhil dissertation, University of Hyderabad.

Maurya, Srivatsa, Petricek and Kochmar. 2025. Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors (MRBench). In *NAACL 2025*.

Morioka, Takayama and Kajiwara. 2026. A dialogue system for L2 learning with dynamic adaptation to in-dialogue difficulty changes. In *SIGDIAL 2026*.

Nagata et al. 2021. Shared task on feedback comment generation for language learners. In *INLG 2021*.

Nagata, Hagiwara, Hanawa and Mita. 2023. A report on FCG GenChal 2022. In *INLG 2023*.

Ogan, Yarzebinski, De Roock, Dumdumaya, Banawan and Rodrigo. 2017. Proficiency and preference using local language with a teachable agent. In *AIED 2017*, LNCS 10331.

Open Spaced Repetition. n.d. py-fsrs: Python implementation of the FSRS scheduler. https://github.com/open-spaced-repetition/py-fsrs.

Pavlik, Cen and Koedinger. 2009. Performance factors analysis: A new alternative to knowledge tracing. In *AIED*.

Pelánek. 2016. Applications of the Elo rating system in adaptive educational systems. *Computers & Education* 98:169–179.

Pon Subbaiah and Balakumar (eds.). 2003. *Tests of Language Proficiency: Telugu*. Mysore: CIIL.

Pon Subbaiah and Balakumar (eds.). 2005. *Tests of Language Proficiency: Hindi*. Mysore: CIIL.

Radford, Kim, Xu, Brockman, McLeavey and Sutskever. 2023. Robust speech recognition via large-scale weak supervision (Whisper). In *ICML*.

Ramanarasimham, P. 2006. *An Intensive Course in Telugu*. Mysore: CIIL.

Reading.com. n.d. Early-reading app; reviewed in the team's SETU proposals.

Ribeiro-Flucht et al. 2026. From dialogue to learner modeling: Productive use in LLM-based grammar practice. In *BEA 2026*.

Ribeiro-Flucht, Chen and Meurers. 2025. A framework for proficiency-aligned grammar practice in LLM-based dialogue systems. In *BEA 2025*.

Roşcan and Nisioi. 2025. Archaeology at TSAR 2025: Teaching small models to do CEFR simplifications. In *TSAR 2025*.

Skidmore, Felice and Dunn. 2025. Transformer architectures for vocabulary test item difficulty prediction. In *BEA 2025*.

Smeaton, Dey-Plissonneau, Lee, Liu and Scriney. 2022. Analysis of individual conversational volatility in tandem telecollaboration for second language learning. arXiv:2206.13965.

Song et al. 2024. GEE! Grammar error explanation with large language models. In *Findings of NAACL 2024*.

Srikanth. 2026. Kelvi: A morphological parser to support Tamil literacy. In *BEA 2026*.

Srivatsa et al. 2025. Can LLMs reliably simulate real students' abilities in mathematics and reading comprehension? In *BEA 2025*.

Stahl and Wachsmuth. 2023. Identifying feedback types to augment feedback comment generation. In *INLG 2023*.

Sundaram, Rao and Sobha L. 2024. LangBot – Language learning chatbot. In *ICON 2024*.

TA review of the SETU proposal. 27 October 2026. Recorded feedback session, IIIT Hyderabad.

Token Guessers. 2026. SETU final proposal; Setu Telugu Tutor: architecture and product proposal. Course documents, Language Models and Agents, IIIT Hyderabad.

U.S. Department of Education. 2016. Non-regulatory guidance: Using evidence to strengthen education investments (ESSA evidence tiers).

Vainikko et al. 2025. Paragraph-level error correction and explanation generation: Estonian. In *BEA 2025*.

Venkateswara Sastry, J. 1980. *Hindi–Telugu Common Vocabulary*. Mysore: CIIL.

Vijayalakshmi, S. A study on the learning process of Hindi by college students with reference to Coimbatore district. PhD thesis, Bharathiar University.

Woo, Rao, Keluskar and Chen. 2026. Evaluating adaptive personalization of educational readings with simulated learners. In *BEA 2026*.

Wyner, G. 2014. *Fluent Forever: How to Learn Any Language Fast and Never Forget It*. New York: Harmony Books.

Yang et al. 2025. HSKBenchmark. arXiv:2511.15574.

Yang et al. 2026. StudentSim: Training LLM-based student simulators. arXiv:2609.01591.

Yang, Lin, Shen, Sheng, Qu and Chen. 2026. TACT: Taxonomy-aligned post-training for pedagogically adaptive English tutoring. arXiv:2608.03952.

Yang, Liu and Wulff. 2025. Using NLI to identify potential collocation transfer in L2 English. In *BEA 2025*.

Zhang et al. 2023. Assisting language learners: Trans-lingual definition generation. In *BEA 2023*.